



哈爾濱工業大學(深圳)

HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

Anemone

设计文档

参赛队名 斗世界电子开发部

队伍成员 张正翰、陈函申

指导老师 夏文、仇洁婷

2026 年 8 月 12 日

摘要

Anemone 是一个使用 Rust 实现、支持 RISC-V64 与 LoongArch64 平台的宏内核。

在开发过程中，我们始终避免为了对特定测例进行特化适配而妥协系统设计。Anemone 的目标是：在 Linux ABI 兼容性、进程线程管理、虚拟内存、VFS 与文件系统、设备驱动模型、IPC、同步和体系结构适配等核心能力域上形成**可解释、可维护、可持续演进**的系统实现。

Anemone 的开发全程独立自主。在架构上，我们抛弃了历史包袱，完全从零设计。我们还编写了基于侵入式结构的 buddy 内存分配器以解决页分配器对堆反向依赖的问题，以及同样基于侵入式结构的、对堆无依赖的设备树解析库以便内核早期使用，以及 LoongArch64 硬件支持库等，它们都独立于且被抽离了内核，形成单独的 crate，经过了 Miri、cargo-fuzz 等工具的验证，我们期望这些 crate 能为未来的开源社区贡献更多的基础设施。

截至本文档编写时，Anemone 已经通过所有决赛测例，并在 Buildstorm 编译测例上取得了可观的性能成绩。

比赛提交到排行榜更新有20秒左右的延迟

#	用户名	队伍	学校	提交次数 (ASC)	buildstorm-glibc-la	buildstorm-glibc-rv	cagent-glibc-la	cagent-glibc-rv	rank
1	T2026104879910415	维塔立体	华中科技大学	46	180.0	180.0	199.1	199.1	758.2000
2	T202610617999569	每天睡满八小时	重庆邮电大学	60	180.0	180.0	199.1	199.1	758.2000
3	T202612332999758	这行代码有问题	烟台南山学院	76	180.0	180.0	199.1	199.1	758.2000
4	T2026104869910383	洗剪吹	武汉大学	77	180.0	169.3	199.1	199.1	747.5000
5	T202610112999587	MyGO!!!!	太原理工大学	34	180.0	168.2	199.1	199.1	746.4000
6	T202610459999616	PulseOS	郑州大学	19	180.0	151.3	199.1	199.1	729.5000
7	T2026181239910993	Unicus	哈尔滨工业大学 (深圳)	36	168.5	157.9	199.1	199.1	724.6000
8	T2026104869911460	最小可运行	武汉大学	28	180.0	140.2	199.1	199.1	718.4000
9	T2026104869910054	猛火烧烤队	武汉大学	51	180.0	138.9	199.1	199.1	717.1000
10	T202610486999578	NexusCore	武汉大学	52	180.0	132.0	199.1	199.1	710.2000
11	T202610486999873	我依旧是世一队	武汉大学	19	180.0	131.4	199.1	199.1	709.6000
12	T2026104869910397	CoreCraft	武汉大学	22	161.8	149.2	199.1	199.1	709.2000
13	T2026103369910596	CosmOS	杭州电子科技大学	49	152.6	112.0	199.1	199.1	662.8000
14	T2026181239910972	斗世界电子开发部	哈尔滨工业大学 (深圳)	44	134.4	60.0	199.1	199.1	592.6000
15	T2026103369910030	拼好核	杭州电子科技大学	62	140.0	20.0	199.1	199.1	558.2000

图 1 Anemone 当前榜单截图。

Anemone 各个模块完成情况概览如下。

模块	完成情况
进程管理	实现 task / thread group / process group 等执行实体管理, 覆盖 fork / clone / exec / exit / wait 等生命周期路径。
调度	围绕 scheduler、wait-core、signal interruption 形成阻塞、唤醒与可中断等待路径。
内存管理	实现地址空间、页表、缺页处理、匿名页、VMO / backing object、file-backed mapping、共享内存与内存压力相关路径。
IPC	覆盖 signal、pipe、System V IPC、event/timer 类文件对象、poll/select 等等待组合路径。
文件系统	实现 VFS、路径查找、mount view、opened file object、procfs、devfs 和多类文件后端的统一接入。
设备驱动模型	实现设备发布、字符/块设备、devfs bridge、ioctl 分发和若干具体设备对象。
网络栈	实现 IPv4 TCP / UDP / ICMP raw、VirtIO-Net、Socket、poll / epoll 接入与只读 Netlink 诊断, 可运行包管理器、HTTPS Git 和原生 iproute2 查询。
时间	围绕 clock、tick、IRQ / threaded soft timer、timerfd 和 itimer 组织时间线、超时与定时通知。
架构硬件抽象层	支持 RISC-V64 与 LoongArch64 的启动、trap、中断、上下文保存和平台差异收束。
构建系统	以类型化配置和统一编排闭合内核、应用、rootfs、设备树、QEMU 与实机产物, 支持双架构、多平台和端到端验证。

表 1 Anemone 模块完成情况概览

摘要	I
1 概述	1
1.1 Anemone 整体架构	1
1.2 项目结构	1
1.3 开发人员分工	3
2 进程管理	4
2.1 不选择无栈协程的理由	4
2.2 进程与线程模型	4
2.3 生命周期路径	7
2.4 Unix 作业控制	8
2.5 资源继承与隔离	9
2.6 ProcFs	9
3 调度	11
3.1 调度状态机	11
3.2 调度循环	12
3.3 多调度类共存	13
3.4 Event 与条件等待	14
3.5 Latch 与 I/O 多路等待	15
3.6 多核分配与负载均衡	15
4 内存管理	17
4.1 物理页管理	17
4.2 内核堆	18
4.3 分页	18
4.4 用户地址空间	19
4.5 ELF 按需装载	21
4.6 VMO 与写时复制	21
4.7 TLB 与多核地址空间	23
4.8 OOM Killer	23
5 IPC	24
5.1 信号机制	24
5.2 Pipe	25
5.3 共享内存	26
5.4 Event 类对象	27
5.5 Epoll	27

5.6	Unix Socket	27
6	文件系统	29
6.1	VFS 对象模型	29
6.1.1	FileSystemOps	30
6.1.2	SuperBlock	30
6.1.3	Mount	30
6.1.4	Inode	31
6.1.5	Dentry	32
6.1.6	File	33
6.2	挂载树	34
6.3	VMO 形式的页缓存	35
6.4	Ext4 支持	36
7	设备驱动模型	37
7.1	核心对象	37
7.1.1	Bus	39
7.1.2	Device	40
7.1.3	Driver	40
7.2	字符设备与块设备	40
7.3	TTY 与交互式终端	42
7.4	设备树与平台初始化	42
7.5	从虚拟机到真实开发板	43
7.6	Devfs 桥接	43
8	网络栈	45
8.1	Socket 能力全景	45
8.2	从调研到四层架构	47
8.2.1	Command、Event 与 Snapshot	49
8.3	帧路径：资源身份停留在设备侧	50
8.4	网络域与 IPv4 控制面	53
8.5	上边界：协议事实如何成为 Linux readiness	54
8.6	后端替换边界	57
8.7	深入真实协议路径的主机测试	57
8.8	只读网络诊断面	60
8.9	真实工作负载闭环	61
9	时间	62

9.1	时间线与 tick	62
9.2	软定时器	62
9.2.1	IRQ	63
9.2.2	Threaded	63
9.3	POSIX Clock	64
9.4	用户可见定时对象	64
10	架构抽象层	65
10.1	启动流程	66
10.2	抽象层	67
10.2.1	PagingArch	67
10.2.2	IntrArch	69
10.2.3	SchedArch	69
10.2.4	TrapArch	70
10.2.5	SignalArch	72
10.3	用户访问与架构异常恢复	73
10.4	架构专属能力	73
11	构建系统	74
11.1	从隐式脚本到显式系统模型	74
11.2	单一入口与统一解析	75
11.3	从配置到可启动系统	76
11.4	双架构与端到端交付	77
12	总结与展望	78
12.1	主要工作成果	78
12.2	开发经验总结	78
12.3	未来计划	79
A	AI 使用情况	80
A.1	使用范围	80
A.2	生成内容范围	81
A.3	人工审查与验证	81
A.4	幻觉与纠正案例	82
A.5	方法论反思	82
B	参考资料	84
B.1	外部参考	84
B.2	往届内核	84

1 概述

Anemone 是一个多核、支持多架构的类 Unix 操作系统内核，目标是在不引入大量历史包袱和污染内部实现的情况下，尽可能兼容 Linux UAPI，从而支持现有的用户态应用程序和工具链。Anemone 具备强大的灵活性和扩展性——我们尽可能避免内核内部的硬编码，而是将各种常量、配置项转移到了可配置的 Kconfig 中，并提供了丰富的内核参数（例如内核抢占，各种文件系统驱动等）和调试接口，方便用户和开发者进行定制化和调试，同时也有利于我们轻松移植到新的架构或平台。目前，Anemone 已经支持 RISC-V64 和 LoongArch64 两种架构，并适配了对应的 QEMU 虚拟平台。

1.1 Anemone 整体架构

Anemone 分成三层：最底层是架构与平台接入层，负责内核自举、trap/中断、上下文保存、时钟中断、机器适配、平台设备发现等；接着是基础设施层，这里包括内核的各个核心子系统：进程管理、调度、时间、内存管理、文件系统、网络、IPC、设备驱动模型等；最后是面向用户的系统调用层，在这里，我们兼容 Linux 的 UAPI，并将其映射到 Anemone 自己的内部对象，避免 Linux 的语义侵入我们的内核，从而造成污染。

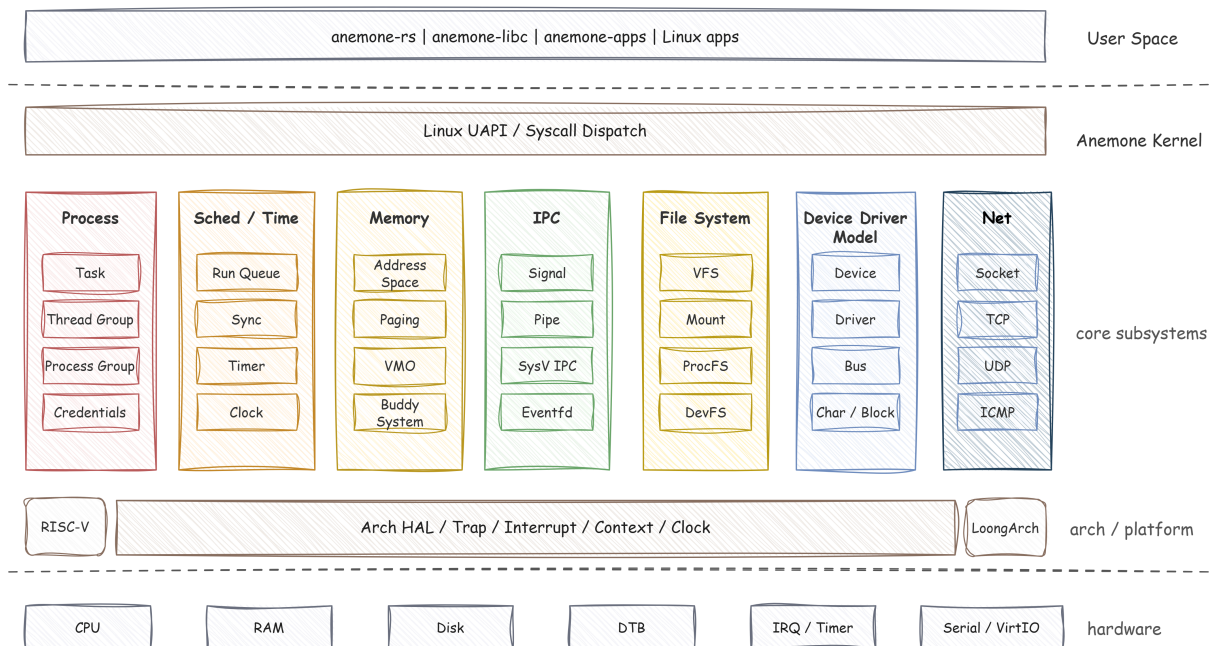


图 2 Anemone 整体架构图。

1.2 项目结构

Anemone 采用单仓库组织内核、共享 ABI、用户态组件、构建配置和工程文档。仓库根目录的 `Justfile` 提供统一入口，复杂的配置解析、构建和运行流程由 `scripts/xtask`

承载；内核与用户态则通过 `anemone-abi` 中的共享类型和系统调用编号保持一致。重要目录如下所示。

```

.
├── anemone-kernel/           # 内核主体与内核侧独立 crate
├── anemone-abi/             # 内核和用户态共享的 ABI 定义
├── anemone-rs/              # Rust 用户态运行时与系统接口
├── anemone-libc/            # C 用户态支持库
├── anemone-apps/            # init、测试、工具和示例程序
├── conf/
│   ├── system-targets/      # 系统镜像与启动目标组合
│   ├── platforms/          # 机器、设备树和 QEMU 平台描述
│   ├── kconfs/              # 可复用的内核 Kconfig 配置
│   ├── build-presets/       # 目标、Kconfig 与构建 profile 组合
│   └── rootfs/              # 根文件系统清单
├── scripts/xtask/           # 配置、构建、rootfs 与 QEMU 编排
├── scripts/                  # 端到端测试和专项验证脚本
├── symtab/                  # 内核符号表生成工具
├── docs/                    # RFC、当前契约、开发记录与问题登记
├── report/                  # 技术报告与展示材料
├── xref/                    # 固定版本的外部实现参考
├── Cargo.toml               # 内核工作区及内部 crate 组成
└── Justfile                 # 仓库统一操作入口

```

代码 1 Anemone 顶层目录

`anemone-kernel/src` 按内核职责拆分。系统调用层负责 Linux ABI 的解析、用户指针访问和参数转换，核心子系统拥有内部状态与生命周期，架构及驱动代码负责把这些机制接到具体 CPU 和设备。主要模块如下。

```

anemone-kernel/
├── src/
│   ├── arch/                # RISC-V64、LoongArch64 架构实现
│   ├── exception/          # trap、异常、中断与 IPI 入口
│   ├── syscall/            # syscall 注册、分发和用户访问边界
│   ├── task/               # 任务、线程组、信号、凭据与资源
│   ├── sched/              # 调度类、运行队列、等待与负载均衡
│   ├── time/               # clock、tick、timer 与 POSIX 时间机制
│   ├── mm/                 # 物理页、页表、地址空间与缺页处理
│   ├── fs/                 # VFS、mount、伪文件系统与文件后端
│   ├── net/                # 网络协议栈接入和网络工作线程
│   ├── device/             # 设备、总线、发现流程与 I/O 对象
│   ├── driver/             # 中断控制器、存储、串口、网卡等驱动
│   ├── sync/               # 锁、事件和内核同步原语
│   ├── debug/              # 日志、KUnit 与性能观测设施
│   └── uts/                # 系统身份与 UTS namespace
├── crates/
│   ├── anemone-net-api      # 内核与协议栈之间的窄接口
│   ├── anemone-smoltcp-stack # smoltcp 网络栈适配层
│   ├── buddy-system        # 伙伴页分配器
│   ├── device-tree         # 设备树解析
│   ├── idalloc              # 标识符分配
│   ├── range-allocator     # 区间分配
│   ├── kernel-macros       # 内核过程宏
│   ├── la-insc              # LoongArch 指令支持
│   └── anemos/              # 内核使用的外部 crate 适配版本

```

代码 2 Anemone 内核主体目录

1.3 开发人员分工

- 张正翰：进程管理，内存管理，文件系统，IPC，网络栈，设备驱动，RISC-V 架构适配，时间管理，syscall 实现，文档撰写，测例支持。
- 陈函申：PCIe 总线，进程管理，内存管理，LoongArch 架构适配，物理开发版移植，测例支持。

2 进程管理

2.1 不选择无栈协程的理由

Anemone 没有把进程管理建立在无栈协程上，而是让每个 `Task` 拥有独立的内核栈和上下文切换状态。调度器切换的是完整的内核执行上下文，而不是把一个系统调用拆成多段手工状态机。

我们调研往届内核时看到过不少内核都选择了无栈协程方案。它的优点是单个执行单元占用内存更小，部分路径上的切换成本也低；不过，就内核实现而言，它会带来三个实际问题。首先，内核调用栈被拆散后，panic、日志和调试信息很难直接还原阻塞点，必须额外引入结构化的 trace 机制，这导致调试相当困难。第二，系统调用和页错误处理这类路径天然会跨越 VFS、内存管理、信号、等待队列等模块；如果每个阻塞点都改写成状态机，代码会在错误处理和资源回滚上变得分散，分析会比较困难。最后，无栈协程本质上是协作式模型，可以用时钟中断抢占用户态，但没办法实现内核态抢占。

Anemone 最后选择有栈调度，我们支持内核抢占——这带来了更高的实时性与响应能力。同时另外两个困难的解决也使得我们在分析数据/控制流时更简单。

2.2 进程与线程模型

Anemone 使用单一的 `Task` 表达可运行的执行上下文，而不是分成 `Process` 和 `Thread` 两套结构，这套设计来自 Linux 的 LWP。这使得各种资源的共享和独占关系的表达力更强：我们可以使用 `Arc` 句柄在 `Task` 之间共享地址空间、文件系统上下文、文件描述符表和信号处置表；也可以在 `Task` 内部使用普通的 `Box` 或 `RwLock` 独占这些资源。通过这种细粒度的共享，我们可以实现 fork/clone/exec 的各种组合，支持多线程进程、单线程进程和内核线程。但客观而言，这样做也存在一个缺点，那就是并发与锁的分析难度会大大上升。这是一种权衡：我们选择了更强的表达力，代价是更复杂的并发分析，而不是更简单的模型。

用户态看到的进程、线程组、进程组和会话由 `TaskTopology` 维护；调度器看到的是 `Task`。这和 Linux LWP 的思路一致：线程和进程在执行上下文层面都是 task，差别来自它们共享或独占哪些资源，以及它们被挂到哪个拓扑关系里。

```

pub struct Task {
    tid: NoIrqRwLock<TidRef>,
    creator: Option<Tid>,
    tgid: Tid,
    create_instant: Instant,

    kstack: KernelStack,
    name: NoIrqRwLock<Box<str>>,
    flags: NoIrqRwLock<TaskFlags>,
    usp: RwLock<Option<Arc<UserSpaceHandle>>>,

    cpuid: CpuId,
    nice: AtomicIsize,
    sched_ctx: MonoFlow<TaskContext>,
    sched_entity: SpinLock<SchedEntity>,
    fpu_used: AtomicBool,

    fs_state: Arc<RwLock<FsState>>,
    files_state: RwLock<Arc<RwLock<FilesState>>>,
    cred: RwLock<CredentialSet>,
    no_new_privs: AtomicBool,
    cpu_usage: NoIrqRwLock<TaskCpuUsage>,

    sig_disposition: Arc<NoIrqRwLock<SignalDisposition>>,
    sig_mask: NoIrqSpinLock<TaskSigMaskState>,
    sig_pending: NoIrqSpinLock<PendingSignals>,
    sig_altstack: NoIrqSpinLock<Option<SigAltStack>>,

    robust_list: SpinLock<Option<VirtAddr>>,
    exit_code: SpinLock<Option<ExitCode>>,
    vfork_done: Event,
    sched_state: NoIrqRwLock<TaskSchedState>,
    clear_child_tid: SpinLock<Option<VirtAddr>>,
    kthread: SpinLock<Option<KThreadTaskLocal>>,
}

```

代码 3 Task 作为 TCB 保存执行上下文、资源句柄、信号状态和退出状态

Task 内只保存当前任务的“本地”状态。tid 和 tgid 标识当前执行上下文及其线程组；kstack、sched_ctx、sched_entity 和 sched_state 服务调度与上下文切换；usp 指向用户地址空间；fs_state 和 files_state 分别描述工作目录、根目录、umask 与文件描述符表；cred 和 no_new_privs 参与权限检查；信号处置表、信号屏蔽字、task-local pending signal、alt stack 和 robust futex list 由 signal、futex、exec 和 exit 路径使用。

从这里可以看出，我们完全没有将拓扑关系放在 TCB 里，它们在全局拓扑表（一个单例）里维护：

```
struct TaskTopology {
    inner: NoIrqRwLock<TaskTopologyInner>,
}

struct TaskTopologyInner {
    tasks: BTreeMap<Tid, TaskNode>,
    thread_groups: BTreeMap<Tid, Arc<ThreadGroup>>,
    process_groups: BTreeMap<Tid, Arc<ProcessGroup>>,
    sessions: BTreeMap<Tid, Arc<Session>>,
}

static TOPOLOGY: TaskTopology = TaskTopology {
    inner: NoIrqRwLock::new(TaskTopologyInner {
        tasks: BTreeMap::new(),
        thread_groups: BTreeMap::new(),
        process_groups: BTreeMap::new(),
        sessions: BTreeMap::new(),
    }),
};
```

代码 4 TaskTopology 保存 task、thread group、process group 和 session 的全局索引

tasks 提供 TID 到 Task 的查找；thread_groups、process_groups 和 sessions 分别维护 Linux 可见的 TGID、PGID 和 SID 层次。拓扑读写通过同一把 NoIrqRwLock 线性化，涉及 session、process group 和 thread group 的内部修改时使用固定锁序：TOPOLOGY → Session.inner → ProcessGroup.inner → ThreadGroup.inner。

为何不直接把拓扑关系维护在 Task 里，比如说 Task 内部有某个字段形如 Arc<ThreadGroup>？这主要是为了避免竞态关系。一个 task 正式进入系统主要有两个

阶段：关系发布和全局表注册。如果我们先发布关系，再注册到全局表，那么这中间就会存在一个窗口：其他 task 可以通过拓扑查到这个 task，但是却无法在全局表里找到它；如果我们先注册到全局表，再发布关系，那么系统内就会出现一个孤立于进程树的 task。无论哪种方式，都会导致拓扑关系的不一致。综上所述，我们选择了在同一个拓扑写锁下完成关系发布和全局表注册，这样就保证了拓扑关系的一致性。

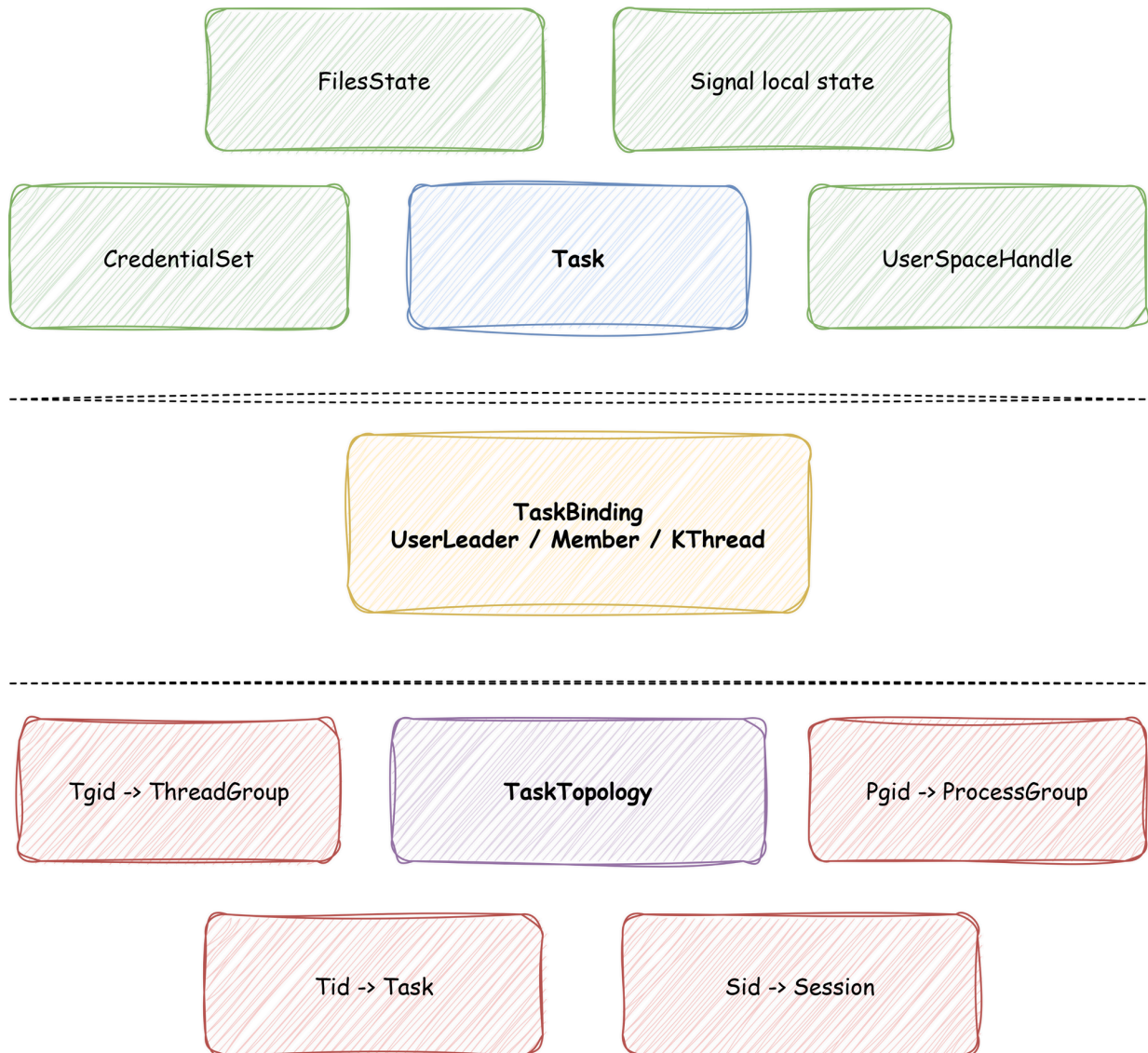


图 3 进程管理全局拓扑。上半部分是单个 Task 持有或引用的执行资源，下半部分是 TaskBinding 发布后进入的全局拓扑索引。

TaskBinding 是新 task 发布到全局拓扑时的分类。UserLeader 创建新的用户线程组，并继承父进程的 PGID、SID 和终止信号；Member 把新 task 加入现有用户线程组；KThread 创建内核线程的 singleton thread group。发布过程在同一个拓扑写锁下完成全局索引更新、线程组更新、父子关系更新以及进程组/会话关系更新，避免用户态观察到半初始化的进程关系。

2.3 生命周期路径

进程创建从 `fork` / `clone` 进入。内核先为子任务准备独立的执行上下文，使它从系统调用返回时在子进程视角中看到返回值为 0；随后根据 `clone` 标志决定哪些资源共享、哪些资源复制。普通 `fork` 会产生新的线程组，子进程继承父进程的进程组和会话；线程创建则加入父进程已有的线程组。子任务只有在地址空间、文件描述符表、文件系统上下文、信号状态和 `credentials` 都初始化完成后，才会一次性发布到全局拓扑并进入调度队列。这样可以避免一个还没有完整资源视图的任务被其他 CPU 或 `procfs` 提前观察到。

程序替换由 `execve` 完成。它不会创建新的进程身份，而是在当前线程组内装入新的程序映像：路径解析使用当前进程的根目录和工作目录，装载器建立新的用户地址空间、用户栈和入口点，并根据 `setuid/setgid` 和 `file capability` 重新计算执行后的权限状态。`exec` 成功后，旧程序的用户态执行现场被替换，自定义信号处理、`alt stack`、`robust futex` 状态以及带 `close-on-exec` 标记的文件描述符被清理；多线程进程执行 `exec` 时，线程组会做“`dethread`”，这将杀死所有其他线程，只保留调用 `exec` 的线程继续执行，而它若非 `leader`，也会被提升为新的线程组 `leader`。

退出路径区分单个线程退出和整个线程组退出。单个线程退出时，内核要先完成用户态可见的退出副作用，例如清理 `clear-child-tid`、唤醒相关 `futex`、处理 `robust futex list`、关闭本线程组不再需要的文件描述符，并从线程组成员集合中移除当前线程，等等。而如果退出的是线程组最后一个成员，内核会把线程组变成可等待的 `zombie` 状态，记录退出码和资源使用情况，将孤儿子进程重新挂到 `init` 下，并通知父进程。

等待路径对应 `wait4` / `waitid` 等接口。父进程可以等待任意子进程、指定 `TGID` 的子进程，或指定进程组内的子进程。除了回收已经退出的 `zombie`，决赛阶段的实现还可以报告子进程的停止和继续状态，并通过 `wait4` 返回资源使用量。这样，`shell` 既能等待普通前台命令结束，也能跟踪被挂起或重新恢复的作业。

这些基本动作，就构成了 Anemone 进程管理的核心生命周期路径。实际上这也是类 Unix 内核的通用做法。

2.4 Unix 作业控制

进入决赛阶段后，我们在原有进程组、会话和信号机制之上补齐了 Unix 作业控制主路径。一个交互式 `shell` 可以把一组相关进程放进同一个 `process group`，将其中一个组设为终端前台作业，并通过 `SIGTSTP`、`SIGSTOP`、`SIGCONT` 等信号停止或恢复整个作业。父进程再通过 `wait4` / `waitid` 获得 `stopped`、`continued` 和 `exited` 状态，从而实现 `jobs`、`fg`、`bg`、`Ctrl-C` 和 `Ctrl-Z` 等常见交互。

这项能力看起来像是若干系统调用和信号的组合，实际上贯穿了进程拓扑、调度和终端三个子系统。我们让线程组保存作业整体的运行、停止和继续状态，调度器负责确保停止后的线程不再返回用户态，TTY 则只负责判断前后台关系并生成相应信号。这样，一

个多线程进程仍然作为一个完整作业被观察，而不会出现某些线程已经停止、另一些线程却继续执行的混乱状态。

作业控制也是 Anemone 从“能启动程序”走向“能运行交互式 Unix 用户环境”的重要一步。BusyBox `ash` 可以使用真实 `controlling terminal` 管理前台进程组，终端输入产生的信号会沿 `process group` 正确投递，父进程也能获得后续状态变化。这套机制没有为 `shell` 编写特判，而是由通用的 `topology`、`signal`、`wait` 和 `TTY` 能力共同完成。

2.5 资源继承与隔离

资源继承主要由 `clone` flag 决定。默认的 `fork` 形态会复制地址空间、文件系统上下文、文件描述符表和信号处置表；带共享 flag 的 `clone` 则让子 task 直接持有父 task 的对应 Arc 句柄。TCB 中的 `usp`（用户空间指针）、`fs_state`（文件系统状态）、`files_state`（打开的文件描述符）和 `sig_disposition`（信号处置表）都按这个规则初始化，因此共享关系在对象引用层面表达，而不是通过额外的进程结构体转发。这也是 LWP 机制的体现。

权限状态在 `fork/clone` 和 `exec` 边界上的处理不同。创建子进程时，子进程继承父进程的用户 ID、组 ID、能力集、`no_new_privs` 和调度优先级等属性；这符合“子进程先从父进程复制而来”的语义。`exec` 时则要重新检查被执行文件的权限属性：`setuid/setgid` 位可能改变 effective ID，`file capability` 可能改变能力集，`no_new_privs` 会禁止通过 `exec` 获得新的权限。换句话说，`fork/clone` 复制身份，`exec` 重新计算身份。

`exec` 的资源边界也不同于 `clone`。它保留进程身份、线程组关系、工作目录、根目录以及未设置 `close-on-exec` 的打开文件；同时替换用户地址空间，重置信号处置中的用户自定义处理函数，清除只属于旧程序映像的用户态辅助状态，并关闭需要在 `exec` 边界消失的文件描述符。这样，`fork/clone` 负责建立“新执行实体”，`exec` 负责把“同一个进程身份”切换到新的程序映像。

2.6 ProcFs

ProcFs 是进程管理向用户态暴露状态的主要入口。在 Anemone 中，`/proc/<tgid>` 是按访问懒创建的，不是在 task 创建时预先注册：如果一个 task 对应的 `procfs` 条目未被访问，那么这个条目就根本不会被注册。这样做是因为很多时候很多 task 的 `procfs` 目录项不会被访问，预先注册会浪费内存，也会降低内核性能。

`/proc/<tgid>` 下的条目由一个静态数组 `TGID_ENTRIES` 统一注册。当前我们已经实现了 `root`、`cwd`、`cmdline`、`environ`、`exe`、`fd`、`mounts`、`stat` 和 `status` 等等。每个子 inode 都持有 `ThreadGroupBinding`，操作前，我们仍然会校验当前绑定是否合法——检查这个 `tgid` 是否仍然对应着此前的 task，这样可以避免用户态访问已经退出的进程的 `procfs` 条目。

退出和回收路径负责使 `procfs` 视图失效。在这里我们明确区分了用户线程和内核线程。不论如何，失效操作会接触条目和 `task` 的绑定。因此，`procfs` 目录项的生命周期跟随 `topology` 的活跃性和 `wait/reap` 结果，而不是单独维护一套进程状态。通过这样的设计，我们可以保证 `procfs` 视图的正确性和一致性，同时避免了额外的资源管理开销。

3 调度

调度的核心是 Task 的调度状态机。Anemone 的调度设计没有把“用户态看到的状态”和“调度器内部状态”混成同一个字段，而是让内部状态先给出正确的调度转换，再投影成 Linux 风格的 task state。这样，我们在实现的时候，首先考虑的不是兼容 Linux，而是自洽的逻辑和证明，这种思路为我们提供了很大帮助。

3.1 调度状态机

Anemone 使用 `TaskSchedState` 记录一个 Task 的内部调度状态。它的顶层状态只有三类: `Runnable`、`Waiting`、`Zombie`。等待态内部再记录等待轮次、是否允许被信号打断，以及当前是尚未真正 park 的 `PrePark`，还是已经从运行队列上脱离的 `Parked`。

```
pub enum TaskSchedState {
    Runnable,
    Waiting {
        state: Arc<WaitState>,
        interruptible: bool,
        park: ParkState,
    },
    Zombie,
}

pub enum ParkState {
    PrePark,
    Parked,
}
```

代码 5 `TaskSchedState` 保存调度器内部状态，`ParkState` 区分 wait 发布后到真正 park 之间的窗口

这个状态机的核心作用是把**逻辑完成**和**物理入队**分开。一个 task 开始等待时，状态先从 `Runnable` 变成 `Waiting { park: PrePark }`。如果在它真正调用调度器、准备从 CPU 上切走之前，等待条件已经被其它 CPU 或中断路径满足，那么唤醒方只需要把状态改回 `Runnable`，后续调度器会发现本轮等待已经完成，并把它作为普通可运行 task 继续处理。如果 task 已经进入 `Parked`，唤醒方则在完成逻辑唤醒后再把它放回目标 CPU 的运行队列。

我们之所以要特地引入 `PrePark`，是因为阻塞路径必须先把“我正在等什么”发布出去，事件源才有机会唤醒它；但发布等待状态和真正切出 CPU 之间一定存在窗口。如果把这两个动作当成一个不可分状态，早到的 `wakeup` 很容易变成旧轮次唤醒、重复入队或 `lost wake`。Anemone 让等待轮次拥有独立身份，旧 token 只能完成它所属的那一轮等待，不能误伤下一轮等待。实际上，早期开发过程中，这样的竞态就曾经导致过一个难以复现的 bug：一个 task 在等待信号时被唤醒，但它还没有真正 `park`，结果在 `wakeup` 之后又被调度器再次 `park`，导致它永远无法被唤醒。

如前所述，Anemone 最后还是得考虑兼容 Linux 的语义。比如 `/proc/<pid>/stat`、`/proc/<pid>/status`，它们都有稳定的 Linux UAPI。所以我们使用观察性投影：`Runnable` 映射为 `running`，`Waiting` 根据 `interruptible` 映射为 `interruptible` 或 `uninterruptible sleep`，`Zombie` 映射为 `zombie`。这个投影可以近似 Linux 的 `TASK_RUNNING`、`TASK_INTERRUPTIBLE`、`TASK_UNINTERRUPTIBLE` 和 `TASK_ZOMBIE`。

3.2 调度循环

Anemone 没有像 Linux 那样，不存在一个中间的调度上下文。这的确牺牲了部分性能，但是我们认为这是值得的——我们换来了更清晰的边界。

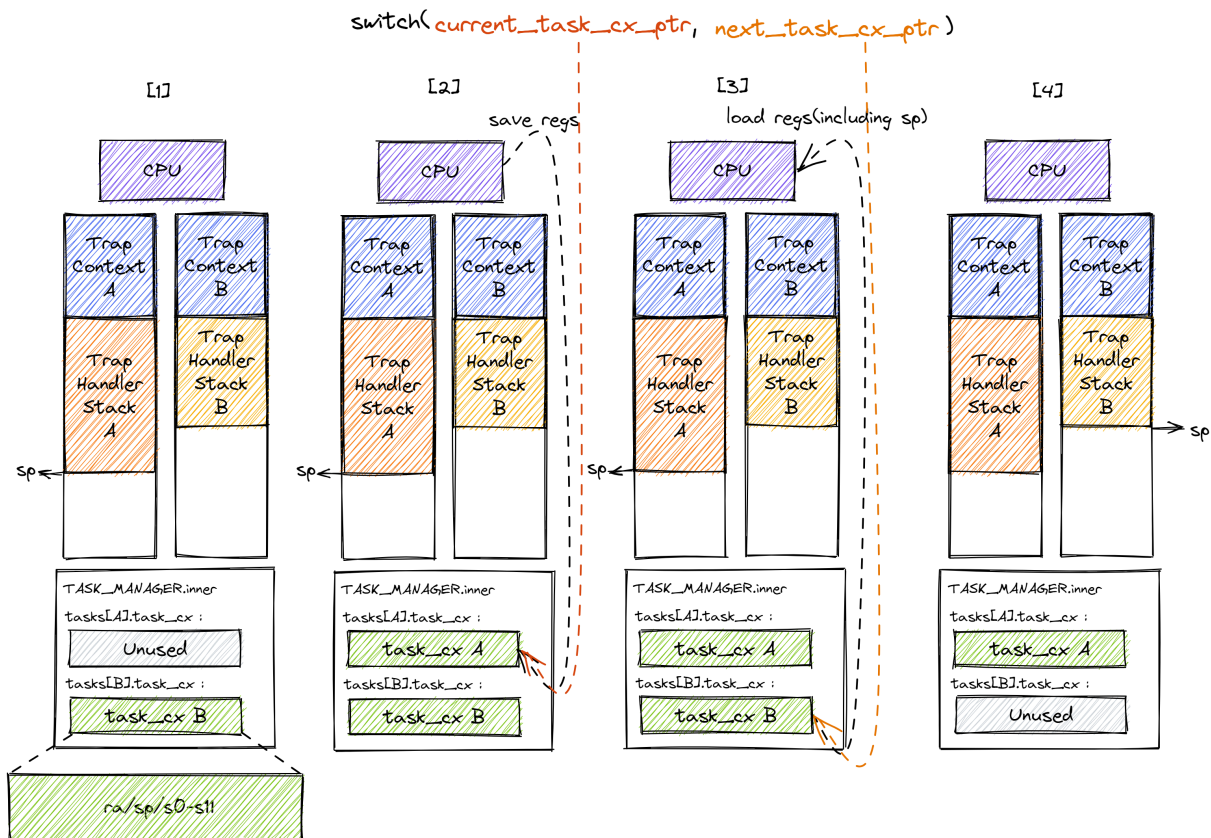


图4 来自 rCore，一致的中心化调度上下文设计

在 Anemone, 每个 CPU 都有自己的调度器上下文、当前运行 task 和本地运行队列。调度循环从本地运行队列选出下一个 task, 切换地址空间与内核执行上下文。

时钟中断负责推进抢占请求。每次 tick 到来时, 本地调度类可以返回 `Resched`, 处理器状态中的 `need_resched` 被置位。用户态 trap 返回前会检查该标志; 如果需要重新调度, 就在返回用户态之前进入调度器。

Anemone 支持内核抢占。它由 `kernel_preempt` 配置控制: 当该配置打开且当前 CPU 允许抢占时, 内核 trap 处理路径也可以在合适位置执行调度。临界区通过关中断或 `PreemptGuard` 禁止这类抢占, 从而保护 per-CPU 数据和短临界区。

与大部分往届内核不太一样的是, 我们没有在时钟中断上下文就立即 `schedule`, 而是把“请求重新调度”和“立刻切换上下文”分开。tick、中断和远程唤醒可以只设置调度请求; 真正切换发生在 trap 返回、显式 `yield`、阻塞等待或调度器安全点。这样既避免在任意锁持有位置强行切换, 又能让长时间运行的用户态程序和可抢占内核路径及时让出 CPU。

3.3 多调度类共存

Anemone 的运行队列借鉴 Linux sched class 的分层思路: 每个 task 持有一个调度实体, 调度实体记录它属于哪个调度类; 每个 CPU 的运行队列按调度类组织 ready task。调度器挑选下一个 task 时, 按照实时、公平和 idle 的优先级寻找下一个执行者, 而每个调度类只维护自己的队列与时间记账。

```
pub struct RunQueue {
    ntasks: usize,
    realtime: Realtime,
    fair: Fair,
    idle: Idle,
}

pub enum SchedClassPrv {
    Realtime(RtEntity),
    Fair(FairEntity),
    Idle(()),
}
```

代码 6 每 CPU 运行队列同时容纳实时、公平和 idle 调度类

普通任务使用基于 Stride 的公平调度。我们把 Linux nice 值映射为权重, 权重越高的任务获得越大的 CPU 份额; 调度器通过 pass 值持续选择当前获得服务最少的任务。与

简单轮转相比，这一机制既保留了实现上的直接性，也能让编译、shell 和后台服务等不同负载按照优先级公平共享处理器。

实时调度类实现了 Linux 风格的 `SCHED_FIFO` 与 `SCHED_RR`。不同实时优先级严格分层，同优先级的 FIFO 任务可以持续运行到主动阻塞、让出或被更高优先级任务抢占；RR 则在此基础上增加时间片轮转。实时类始终位于公平类之前，而当系统没有普通任务可运行时，本地 idle task 才会接管 CPU。

在用户接口上，我们支持 `sched_setscheduler`、`sched_setparam`、`sched_setattr`、`nice` 与 CPU affinity 等常用调度配置。策略切换并不只是修改一个整数：task 可能正在运行队列上，也可能正在某个 CPU 执行，因此配置变化、调度实体变化与重新排队需要作为一次完整操作完成。我们把这些动作收敛到调度器内部，使用户态可以动态调整策略和优先级，而不会让某个 task 同时出现在两个调度类中。

这里值得一提的是，idle 循环也必须检查 `need_resched`，这样即使内核抢占关闭，也能在有新 task 变为 ready 后回到调度器。否则内核会停留在 idle 上下文，不再让出 CPU。对我们而言，多调度类的价值并不只在于接口数量，而在于普通负载、实时任务和空闲处理能够共享同一套状态转换、抢占与等待基础设施。

3.4 Event 与条件等待

`Event` 是 Anemone 中最常用的条件等待原语，功能接近 Linux wait queue。它本身不传递数据，只表达“某个条件可能已经变化”。等待方注册 listener 后必须再次检查 predicate；发布方唤醒 listener 后，等待方醒来也必须重新检查 predicate。因此，Event 的 wakeup 是提示，不是条件已经永久成立的证明。

`Event` 支持可中断等待、不可中断等待、带 timeout 的等待，以及 exclusive / non-exclusive listener。non-exclusive listener 会在一次 publish 中全部尝试唤醒；exclusive listener 则按指定 quota 唤醒，用来避免某些资源只释放一个单位时把所有等待者都叫醒。`Mutex` 的慢路径就是一个代表例子：快速路径先用原子操作抢锁，失败后通过 `Event` 不可中断等待锁释放；解锁时发布一个 exclusive wakeup，让一个等待者重新竞争锁。

`Event` 适合“源对象拥有等待队列”的场景，例如 mutex、futex、pipe close、child exit、vfork completion 等。它不适合直接表达 `poll` / `select` 这类“一次 syscall 同时等待多个源，只要任意一个源 ready 就返回”的场景，因为后者的等待队列不属于某一个长期事件源，而属于本次 syscall。

从一定程度上讲，`Event` 其实是一个弱化版的 Linux wait queue。它没有 Linux wait queue 那样的复杂性和多态性，但也因此更容易理解和使用。这是一种权衡，我们在开发过程中认为，在 Anemone 中，Event 的简单性和可理解性更为重要。

3.5 Latch 与 I/O 多路等待

为了解决 `poll` / `select` 的 OR 等待, Anemone 创造性地实现了一个新同步源于: `Latch`, 意为闸门。 `Latch` 表示一轮的等待: 等待方创建一个不可复制的 `Latch`, 再派生出多个可复制的 `LatchTrigger` 分发给 fd source。任意 source 的第一发有效 trigger 都可以完成本轮等待; 旧 trigger 迟到时, 将不会发生任何效果。

最初, 我们引入 `Latch` 的核心诉求是正确地实现 `ppoll` 和 `pselect6`, 替换早期的 busy polling。这里可以用 pipe 举个例子。VFS 对 pipe 进行轮询时, pipe 会检查读写是否可行; 如果暂时无数据, 就保存对应方向的 trigger; 读写端状态变化或关闭时, pipe 就通过 trigger 唤醒正等待在 latch 上的 task。决赛阶段加入的 `epoll`、TTY 和 Socket 也复用了同一套“读取状态—登记通知—再次确认”的等待思路。

3.6 多核分配与负载均衡

Anemone 是多核内核, 调度器采用“创建时分配 CPU, 运行期固定 CPU”的策略, 这可以减少线程在不同 CPU 之间切换的开销, 提高缓存命中率。每个 task 创建时会选择一个目标 CPU; task 之后只在这个 CPU 的本地运行队列中被调度。跨 CPU 唤醒不是由当前 CPU 直接修改远端运行队列, 而是向目标 CPU 投递异步唤醒请求并发送 IPI, 让目标 CPU 在自己的边界内完成入队。

如果唤醒方 CPU 直接访问目标 CPU 的运行队列, 会破坏 per-CPU 数据的一致性。异步投递让唤醒方不必等待远端立即完成, 也让每个运行队列始终只由所属 CPU 修改; 这对八核编译一类频繁创建、阻塞和唤醒任务的负载尤其重要。

我们实现了负载均衡策略。每当新 task 创建时, 我们会使用负载均衡算法决定将其分配到哪个 CPU 上运行。负载均衡算法会考虑当前各个 CPU 的负载情况, 尽量将新 task 分配到负载较低的 CPU 上, 以提高系统整体的性能和响应速度。

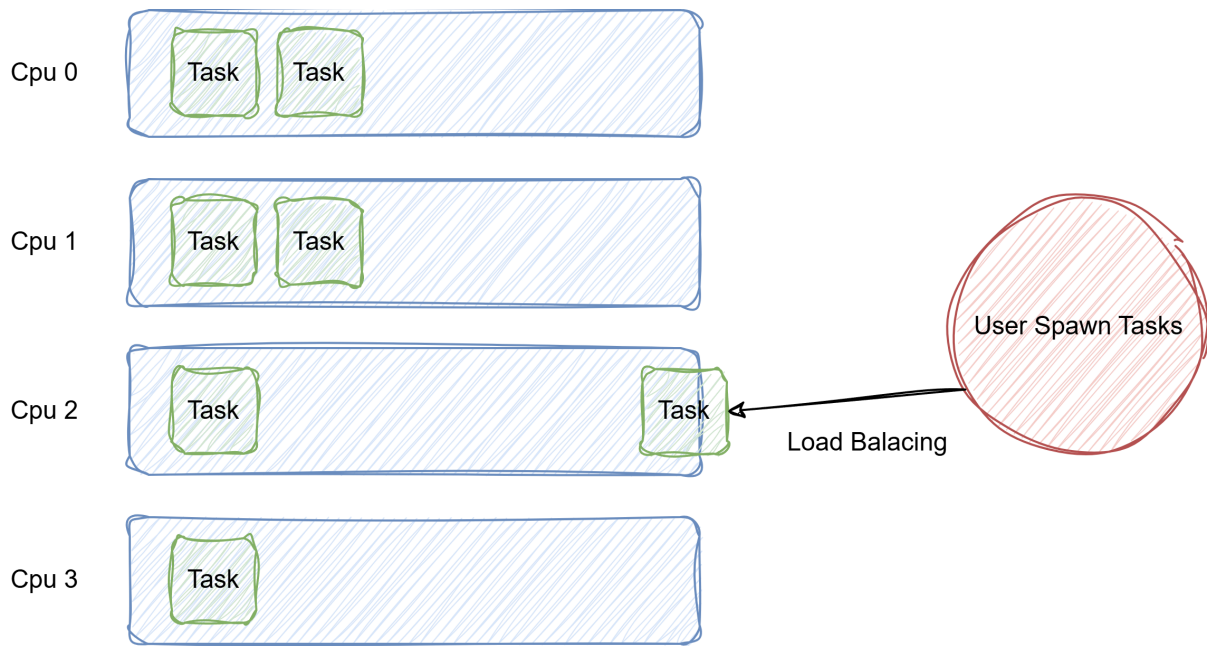


图 5 当前 task 创建时的 CPU 分配与本地运行队列关系。新 task 选择一个目标 CPU 后进入对应运行队列，之后等待唤醒也回到该 CPU。

4 内存管理

Anemone 的内存管理主要可以分为物理页管理、页表映射、用户地址空间几个相对清楚的部分。

4.1 物理页管理

内核启动早期会根据固件提供的内存描述注册可用物理内存区域。物理页分配器接收这些区域后，用 buddy 算法管理连续页分配；上层可以申请单页，也可以申请一段连续页。分配器本身不保存复杂的对象语义，只维护页是否可分配和当前页数统计。

Anemone 实现了单独的 buddy system 库，并经过了 Miri 和 cargo-fuzz 的检验。相比于往届内核常用的 buddy-system-allocator crate，Anemone 的 buddy system 库不再反过来依赖 alloc crate，而是采用侵入式的方式，把页分配器的状态直接嵌入到内核的静态内存中。这样可以避免在内核早期初始化阶段使用 alloc crate 带来的复杂依赖和潜在问题。实际上，促使我们开发这个库的一大原因正是：一开始使用 buddy-system-allocator crate 时，堆内存耗尽时，向页分配器申请页，但页分配器内部又依赖堆，这直接导致了一个“死锁”，实际上是死循环。

页的生命周期由 RAII 句柄约束。FrameHandle 表示对一个物理页的引用，克隆句柄会增加引用计数；句柄释放时减少引用计数，最后一个引用消失后再把页还给 buddy 分配器。OwnedFrameHandle 表示独占页，只有在页没有共享引用时才能取得；需要写入页内容的路径优先使用独占句柄。

```
#[derive(Debug)]
pub struct FrameHandle {
    ppn: PhysPageNum,
}

#[derive(Debug)]
pub struct OwnedFrameHandle {
    inner: FrameHandle,
}
```

代码 7 两种类型内部相同，差别在于暴露的语义

Anemone 参考 Linux 的 memmap 设计，创新性地引入了基于基数树的 memmap 管理。每个物理页可以直接通过其物理页号在 O(1)时间内找到对应的 memmap 结构体。memmap 结构体中包含了该物理页的状态信息，比如是否被分配，是否被锁定等。

同时, `memmap` 结构体还包含了指向该物理页所属的内存区域的指针, 这使得我们可以快速地找到该物理页所在的内存区域, 从而进行更高效的内存管理操作。

```
#[derive(Debug)]
pub struct Frame {
    ppn: PhysPageNum,
    // Reference count
    rc: AtomicUsize,
}
```

代码 8 物理页的元数据

4.2 内核堆

调研往届内核时我们发现, 大部分作品的堆都是固定大小的, 一开始就被嵌入内核的 `bss` 段中。这导致了内核堆的大小无法动态调整, 限制了内核的灵活性。一旦发现堆不够大, 就必须重新编译内核, 或者在运行时进行复杂的内存管理操作。

Anemone 的内核堆是动态可扩展的, 它可以根据需要在运行时进行扩展。当堆内存不够时, 它会向底层页分配器请求更多的物理页, 并将这些页映射到内核堆的虚拟地址空间中。这样, 内核堆的大小可以根据实际需求进行调整, 提高了内核的灵活性和可扩展性。

我们使用 `Talc` 作为全局分配器。启动早期先使用一段 `bootstrap heap`; 当堆空间不足时, 堆的 `OOM handler` 会向物理页分配器申请新的页, 把这些页加入 `Talc` 的可分配 `span`。这样内核堆不需要在链接脚本中固定一个很大的静态上限, 实际使用量由运行时需求推动。

4.3 分页

`PageTable` 持有硬件页表根页, 具体的遍历、映射、解除映射和权限修改由 `Mapper` 完成。这个边界让架构相关代码只需要提供页表项格式、地址空间切换、`TLB` 刷新等硬件能力, 通用内存管理逻辑则复用同一套映射计算。

页表页本身也来自物理页分配器。创建地址空间时, 内核先分配并清零根页, 再复制内核共享映射; 销毁地址空间时, `PageTable` 会解除用户态范围内的映射, 并把页表页按受管页重新交还。这样, 我们就通过 `RAII` 确保了页表页的生命周期和地址空间对象的生命周期一致, 避免了悬空页表页或内存泄漏的问题。

```
/// PageTable. The container of page directories.
///
/// The mapping/unmapping logic is implemented by mappers.
#[derive(Debug)]
pub struct PageTable {
    root: PhysPageNum,
}

/// Mapper. Computation engine for page table traversal and
modification.
#[derive(Debug)]
pub struct Mapper<'a> {
    root: PhysPageNum,
    _lifetime: PhantomData<&'a mut PageTable>,
}
```

代码 9 PageTable 持有页表根页，Mapper 提供映射操作

4.4 用户地址空间

Anemone 中，我们使用 `UserSpace` 表示一个用户进程的地址空间，内部包含页表、按起始虚拟页组织的 VMA 集合、用户栈和堆的保留区，以及 SysV shared memory 的 attachment 记录，等等。新建地址空间时，内核会建立零页 guard、栈保留区和 heap 保留区；真正的物理页通常延迟到第一次访问时再分配，这样可以避免为未使用的内存浪费物理页。

```
#[derive(Debug)]
pub struct UserSpace {
    /// Underlying hardware page table.
    table: PageTable,

    /// User virtual memory areas, including stack and heap.

    vmas: BTreeMap<VirtPageNum, VmArea>,
    /// SysV shared memory attachments by page-aligned attach
    start.

    sysv_shm: BTreeMap<VirtPageNum, shm::ShmAttachment>,

    stack: Stack,

    heap: Heap,

    /// Command-line argument region. [start, start + size).
    Strings, not
    /// pointers.
    ///
    /// /proc/[id]/cmdline needs this.
    cmdline_range: Final<(VirtAddr, usize)>,
    /// Environment variable region. [start, start + size).
    Strings, not
    /// pointers.
    ///
    /// /proc/[id]/environ needs this.
    env_range: Final<(VirtAddr, usize)>,

    ...
}
```

代码 10 用户地址空间对象

4.5 ELF 按需装载

初赛阶段的 ELF 装载会在 `execve` 时把程序段较早地读入内存。进入决赛后，面对 glibc、Rust 工具链和大量动态库，这种方式会带来明显的启动开销和内存浪费：一个程序映像中有相当一部分代码和数据在本次运行里根本不会被访问。

我们把 ELF load segment 改造成了按需分页的 VMO backing。 `execve` 仍然负责检查 ELF 结构、建立地址空间和装载主程序及解释器，但段内容不再全部立即复制；真正访问某一页时，缺页路径才从对应文件页缓存取得内容，并按 ELF 中 file size 与 memory size 的关系补零。只读、页对齐的内容可以直接复用文件页，私有可写段则继续通过 ShadowObject 获得写时复制语义。

这项改造把程序装载自然地接入了已有 VMO 架构，而不是另建一套 ELF 专用缓存。它直接降低了大型程序启动时的磁盘读取和物理页占用，也使动态链接器、编译器和大量短生命周期子进程更适合在 Anemone 上运行。

4.6 VMO 与写时复制

我们认为这是 Anemone 最突出的特征之一：Anemone 没有照搬 Linux，而是参考现代微内核 (Fuchsia/Zircon) 的设计理念，引入了 VMO (Virtual Memory Object, 虚拟内存对象) 架构。

```
pub trait VmObject: Send + Sync {
    /// Resolve the frame at `pidx` for the given access type.
    fn resolve_frame(&self, pidx: usize, access: PageFaultType) →
    Result<ResolvedFrame, SysError>;

    ...
}
```

代码 11 VmObject trait 定义了虚拟内存对象的核心接口

VMO 是用户内存的后端对象。它的核心接口是按对象内页号和访问类型解析一个物理页，并告诉 VMA 这页是否允许以可写权限映射。VMA 不需要知道页来自匿名内存、文件页缓存、共享内存还是 ELF 装载段；它只根据自身权限和 VMO 返回结果安装页表项。

匿名 VMO 在读 fault 上可以返回共享零页，避免为从未写过的匿名页立即分配物理内存；写 fault 才分配并清零一个真实物理页。固定页 VMO 用于 ELF 段等已经准备好的页集合。文件系统 inode 可以暴露自己的 mapping，文件 mmap 因此和普通文件页缓存使用同一类 backing。SysV shared memory 的 VMO 则在第一次访问时物化真实

页，即使第一次访问是读，也必须分配共享页，因为后续其他 attach 者的写入需要通过同一帧可见。这些形形色色的复杂语义被抽象到统一的 VMO 之后，而前端的 VMA 完全不需要关心这些细节！

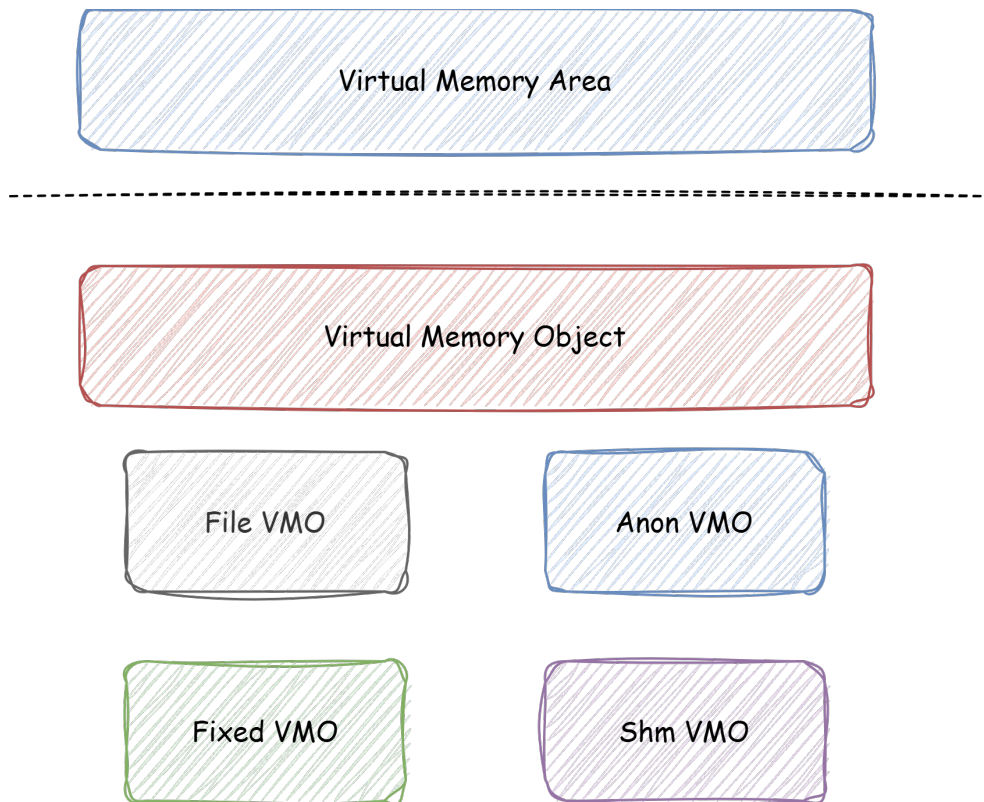


图 6 形形色色的 VMO 被抽象在统一的 VmObject trait 之后

VMO 架构的关键就在于 ShadowObject。它承担私有映射和 fork 后写时复制的核心语义。读和执行访问会沿父对象解析页面，并把返回页降级为不可写；写访问则复制父对象当前页内容，把新页记录在 shadow overlay 中。之后同一页的访问优先命中 overlay，不再影响父对象或兄弟地址空间。通过这个设计，Anemone 自然地实现了 COW、匿名页懒分配与全局只读零页共享，从而大大减少了内核的复杂性和潜在的错误，提高了内存利用率。

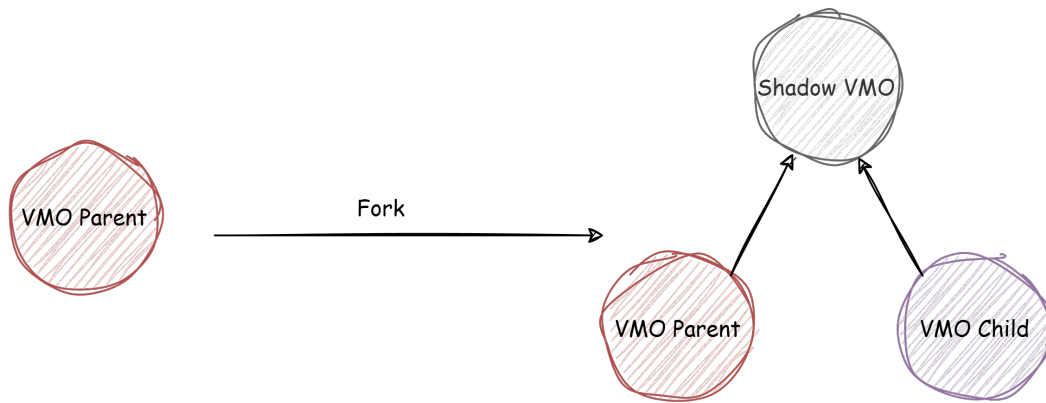


图 7 一个 VMO 经过 fork, 会产生三个新的 VMO

4.7 TLB 与多核地址空间

页表修改之后, 处理器中已经缓存的地址转换也必须同步更新。我们最初采用非常保守的策略: 一旦用户页表发生变化, 就刷新本地 TLB, 并通知所有其它 CPU。它容易保证正确, 却会让缺页、`mmap`、`fork` 和进程退出在多核环境中付出较高代价。

决赛阶段, 我们让页表操作区分“新增映射”和“替换或收紧已有映射”。新的映射不存在旧转换, 普通用户缺页返回时可以利用硬件重新执行指令完成自然装载; 真正可能残留旧转换的修改仍然执行必要的失效。对于远端 CPU, 地址空间会记录哪些处理器仍可能持有自己的转换, 只向这些 CPU 发送 shutdown, 而不是无条件广播到所有核心。

我们还为连续地址范围提供批量失效, 避免逐页发送重复指令或 IPI。经过这些改进, TLB 维护从“每次都做最保守的全局动作”变成了与修改性质和地址空间活动范围相匹配的操作, 既保持双架构上的正确性, 也显著减少了大型编译负载中的额外开销。

4.8 OOM Killer

Anemone 对于 OOM 具备一定的防御能力。在内核启动时, 我们会创建 OOM Killer 内核线程, 并由周期采样检查系统内存水位。一旦可用内存低于可配置阈值, OOM Killer 会寻找占用独占物理页较多的用户进程并发送 `SIGKILL`, 使进程沿正常退出路径释放地址空间和文件资源。

OOM Killer 将在内存水位恢复安全后继续睡眠, 平时不会占用 CPU 空转。配合匿名页懒分配、ELF 按需装载、写时复制以及 `brk` 收缩时的页面回收, 这套机制让 Anemone 能够更从容地承载编译器等内存需求波动较大的真实程序。

5 IPC

Anemone 的 IPC 不只包括传统的信号、管道和共享内存，也包括 eventfd、epoll、Unix Socket 等面向现代事件循环和复杂用户程序的通信机制。我们尽量让这些接口共享文件描述符、阻塞唤醒和 I/O 多路复用基础，而不是为每种对象各写一套等待逻辑。

5.1 信号机制

Anemone 实现了 Linux/POSIX 风格的信号 UAPI，支持 `rt_sigaction`、`rt_sigprocmask`、`rt_sigpending`、`rt_sigsuspend`、`rt_sigtimedwait`、`rt_sigreturn`、`kill`、`tkill`、`tgkill` 和 `rt_sigqueueinfo` 等一系列接口，可以为用户态程序兼容提供强大的支持。用户程序可以设置信号处理函数、修改信号屏蔽字、查询 pending 信号，也可以向进程或指定线程发送信号。

```
pub enum SignalAction {
    Default(fn(SigNo)),
    Ignore,
    Custom(VirtAddr),
}

pub struct KSigAction {
    pub action: SignalAction,
    pub flags: SaFlags,
    pub restorer: VirtAddr,
    pub mask: SigSet,
}

pub enum SigInfoFields {
    Kill(SigKill),
    Rt(SigRt),
    Chld(SigChld),
    Fault(SigFault),
    Timer(SigTimer),
    TKill(SigKill),
    Ill(SigFault),
}
```

代码 12 信号动作、用户可见信号处理配置和 `siginfo` 携带的信息类型

信号动作按照默认处理、忽略和用户自定义 handler 三类保存。每个任务维护自己的信号屏蔽字，线程组和线程定向信号分别进入对应的待处理集合；同步异常、管道写端错误、定时器到期、子进程退出等内核事件也会转化为信号投递给目标任务。这样，普通进程控制和内核异常通知可以复用同一套信号处理路径。

当任务从内核态返回用户态前，内核会检查是否存在未被屏蔽的 pending 信号。如果目标信号使用默认动作，内核直接执行对应的终止、忽略等行为；如果用户注册了 handler，内核会在用户栈上构造信号上下文，保存原来的用户态寄存器和信号屏蔽字，并把返回地址设置到 `rt_sigreturn` 路径。用户态 handler 执行结束后通过 `rt_sigreturn` 回到内核，由内核恢复原始上下文继续执行。

决赛阶段，信号机制进一步成为 Unix 作业控制和 POSIX timer 的共同基础。`SIGSTOP`、`SIGTSTP` 与 `SIGCONT` 可以作用于整个线程组，并由父进程观察停止和继续状态；POSIX timer 则通过携带 timer 信息的实时信号通知目标线程。我们仍然让信号子系统统一负责 pending、mask 和投递，TTY、timer 和 pipe 等事件源只负责产生信号，从而避免不同模块各自实现一套信号语义。

5.2 Pipe

Anemone 通过 `pipe2` 创建匿名管道，并返回读端和写端两个文件描述符。管道内部维护一段环形缓冲区，读端只允许读取，写端只允许写入；文件描述符可以通过 `fork` 继承，也可以被复制到其它 fd，从而让相关进程在同一条字节流上通信。

```

struct Pipe {
    inner: SpinLock<PipeInner>,
    read_operation: Mutex<()>,
    read_recheck: Event,
    write_recheck: Event,
}

struct PipeInner {
    buf: HeapRingBuffer<u8>,
    rx_cnt: usize,
    tx_cnt: usize,
}

struct PipeRx {
    pipe: Arc<SpinLock<Pipe>>,
}

struct PipeTx {
    pipe: Arc<SpinLock<Pipe>>,
}

```

代码 13 Pipe 的核心缓冲区与读写端状态

管道读写遵循 Linux 常见语义：读端在缓冲区有数据时返回实际读取的字节数，所有写端关闭后返回 EOF；写端在读端关闭后返回 `EPIPE`，同时向当前任务投递 `SIGPIPE`。非阻塞 fd 在无法立即完成读写时返回 `EAGAIN`。阻塞读写会在数据、空间或端点状态变化时由 Event 唤醒，不再依赖忙等。管道容量可以动态调整，我们也实现了 `F_GETPIPE_SZ`、`F_SETPIPE_SZ` 和 `FIONREAD` 等常用控制接口。

在匿名管道之外，我们还支持通过 `mkfifo` 创建命名管道。它把同一套字节流和阻塞语义接入 VFS 命名空间，使没有亲缘关系的进程也能通过路径建立通信；打开端点、等待对端、关闭与删除路径则分别沿 VFS 和 pipe 生命周期处理。

5.3 共享内存

Anemone 实现了 System V 共享内存，提供 `shmget`、`shmat`、`shmdt` 和 `shmctl` 等 UAPI。共享内存段可以通过 `IPC_PRIVATE` 创建，也可以通过 key 查找已有对象；每个段都有独立的 `shmid`、权限信息、大小、创建者、最近操作进程和 `attach` 计数。`IPC_RMID` 会先把段标记为删除，等所有进程都 `detach` 后再释放对象。

用户调用 `shmat` 后，内核会把共享内存段接入当前进程地址空间，并在地址空间中记录 attachment。该映射可以在 `fork` 后继承，多个进程访问的是同一个共享 backing，因此一个进程写入的数据能够被其它 attach 者直接看见。共享页由共享内存对象按页管理，和普通匿名内存、文件映射一样经过用户地址空间和缺页处理路径安装到页表中。

除了 System V 接口，Anemone 也支持 `/dev/shm` 形式的文件型共享内存使用方式。比赛测例中，很多测试环境（比如 LTP）会依赖 `/dev/shm` 的共享内存，用户态的 `shm_open`。我们在测试启动之前，在 `/dev/shm` 这里提前挂在一个 tmpfs，这样 `shm_unlink` 和 `mmap` 可以通过普通文件创建、删除和映射来建立共享对象，提供了更强的兼容支持。

5.4 Event 类对象

Anemone 还实现了各类被抽象为 fd 的事件对象。例如：我们实现了 `eventfd2` 系统调用。`eventfd` 用于在进程或线程之间传递轻量级事件计数。`eventfd` 文件内部维护一个 64 位计数器，用户态可以通过读写该 fd 消费或增加计数；创建时支持 `EFD_CLOEXEC`、`EFD_NONBLOCK` 和 `EFD_SEMAPHORE`。在 semaphore 模式下，每次读取只消费一个计数单位，用户可以把 `eventfd` 当作轻量通知或同步原语使用。

5.5 Epoll

为了支持事件驱动程序，我们在 `poll` / `select` 的基础上实现了 `epoll`。一个 `epoll` 实例本身也是文件对象，它保存对目标 opened file description 的关注关系，并把已经就绪的对象组织成可收集的事件。用户可以通过 `epoll_ctl` 增删或修改关注项，再用 `epoll_wait` / `epoll_pwait` 等待多个来源。

`epoll` 并不缓存“这个 fd 永远可读”之类的副本。Pipe、TTY、`timerfd`、`eventfd` 和 Socket 仍然各自拥有真实状态；状态变化时只通知 `epoll` 重新检查。我们复用调度章节介绍的等待机制，使阻塞等待、超时和信号中断可以共同工作。这样，Anemone 能够运行依赖 `epoll` 的 shell 工具、语言运行时和网络程序，而不需要为每种 fd 编写专门的事件循环。

5.6 Unix Socket

我们实现了 `AF_UNIX` stream 与 seqpacket Socket，为同一台机器上的进程提供双向通信。Unix stream 支持 `socketpair`，也支持通过文件系统路径执行 `bind`、`listen`、`connect` 和 `accept`；连接建立后，两端拥有相互独立的收发方向，因此半关闭、EOF、非阻塞 I/O 与 readiness 都能自然表达。Seqpacket 则在同样的本地连接基础上保留消息边界，适合需要记录语义的进程间协议。

Unix Socket 与网络章中的 IPv4 Socket 共用通用的 Socket 前端和 Linux ABI 入口，但不会共享协议私有状态。对用户态来说，它们都可以被 read/write、poll 和 epoll 操作；对内核来说，本地命名空间、连接队列与字节流仍由 Unix Socket 自己维护。这使我们既获得统一的编程接口，也保留了不同通信协议清楚的实现边界。

6 文件系统

Anemone 的文件系统围绕 VFS 对象模型展开：上层系统调用、文件描述符表和内存映射只面对统一的 `File`、`Inode`、`Dentry`、`Mount` 和 `PathRef`，具体后端通过静态 operation table 接入。这样，`Ext4`、`ramfs`、`procfs`、`devfs`、匿名 `pipefs` 等对象可以共享同一套路由、权限检查、路径查找和打开文件语义。

开发过程中，我们发现文件系统与下一章讲的设备驱动模型都非常依赖对象私有数据。所以，我们为此引入 `Opaque trait` 和 `AnyOpaque` 类型（这里的命名来自 Zig 的 `anyopaque` 类型），提供类型安全的类型擦除能力。VFS 对象只保存 `AnyOpaque`，后端在自己的边界内通过 `cast` 或 `cast_mut` 恢复原始类型；这样既不用把 `Ext4`、`procfs`、`devfs` 或设备状态暴露给通用 VFS，也避免了到处使用裸指针传递私有状态。

```
pub trait Opaque: Any + Sync + Send {}

#[derive(Debug)]
pub struct AnyOpaque(Box<dyn Opaque>);

impl AnyOpaque {
    pub fn new<T: Opaque>(data: T) → Self {
        Self(Box::new(data))
    }

    pub fn cast<T: Opaque>(&self) → Option<&T> {
        self.0.cast::<T>()
    }

    pub fn cast_mut<T: Opaque>(&mut self) → Option<&mut T> {
        self.0.cast_mut::<T>()
    }
}
```

代码 14 `Opaque` 与 `AnyOpaque` 提供类型安全的对象私有数据存储

6.1 VFS 对象模型

Anemone 的 VFS 对象采用接近 Linux VFS 的 operation table 形态：通用对象保存静态函数表和后端私有数据，调用时由 VFS 先完成权限、路径、挂载属性和文件光标

等公共检查，再把请求转发给具体文件系统或特殊文件对象。这里我们不使用 trait，让数据全部集中在一个对象上，更加统一。

6.1.1 FileSystemOps

`FileSystemOps` 是文件系统类型的注册入口。每个可挂载文件系统提供名字、属性、挂载函数和同步 / 销毁回调；VFS 根据名字找到 `FileSystem` 后调用 `mount` 生成或复用 `SuperBlock`。

```
pub struct FileSystemOps {
    pub name: &'static str,
    pub flags: FileSystemFlags,
    pub mount: fn(MountSource, MountData) →
Result<Arc<SuperBlock>, SysError>,
    pub sync_fs: fn(&SuperBlock) → Result<(), SysError>,
    pub kill_sb: fn(Arc<SuperBlock>),
}
```

代码 15 `FileSystemOps` 是文件系统类型接入 VFS 的入口

6.1.2 SuperBlock

`SuperBlock` 表示一个已经挂载的文件系统实例。对应的 `SuperBlockOps` 负责按 inode number 载入 inode、在 resident inode cache 回收时写回或取消回收，以及同步 inode 元数据。Ext4、ramfs、procfs 等后端都把自己的 `SuperBlock` 私有状态放在 `AnyOpaque` 中，通用层只维护 `superblock`、`mount` 和 `inode cache` 的关系。

```
pub(super) struct SuperBlockOps {
    pub load_inode:
        fn(&Arc<SuperBlock>, ino: Ino) → Result<Arc<Inode>,
SysError>,
    pub evict_inode: fn(Arc<Inode>) → Result<(), SysError>,
    pub sync_inode: fn(&InodeRef) → Result<(), SysError>,
}
```

代码 16 `SuperBlockOps` 负责 inode 载入、回收和元数据同步

6.1.3 Mount

`Mount` 是挂载树中的视图对象。同一个 `SuperBlock` 可以通过多个 `Mount` 出现在不同路径下，`bind mount` 也通过创建新的 `Mount` 视图来共享原有 `superblock` 和

dentry 树。只读属性保存在 mount 视图上，因此同一后端可以在不同挂载点呈现不同的写入策略。

```
pub struct Mount {
    root: Arc<Dentry>,
    sb: Arc<SuperBlock>,
    placement: SpinLock<MountPlacement>,
    children: SpinLock<Vec<Weak<Mount>>>,
    attrs: AtomicU32,
}
```

代码 17 Mount 保存一次挂载视图的根目录、后端 superblock、位置和属性。可以注意到这里的 `attrs` 是原子字段，这是为了保证挂载点属性修改的原子事务，以免用户态观测到竞态窗口。

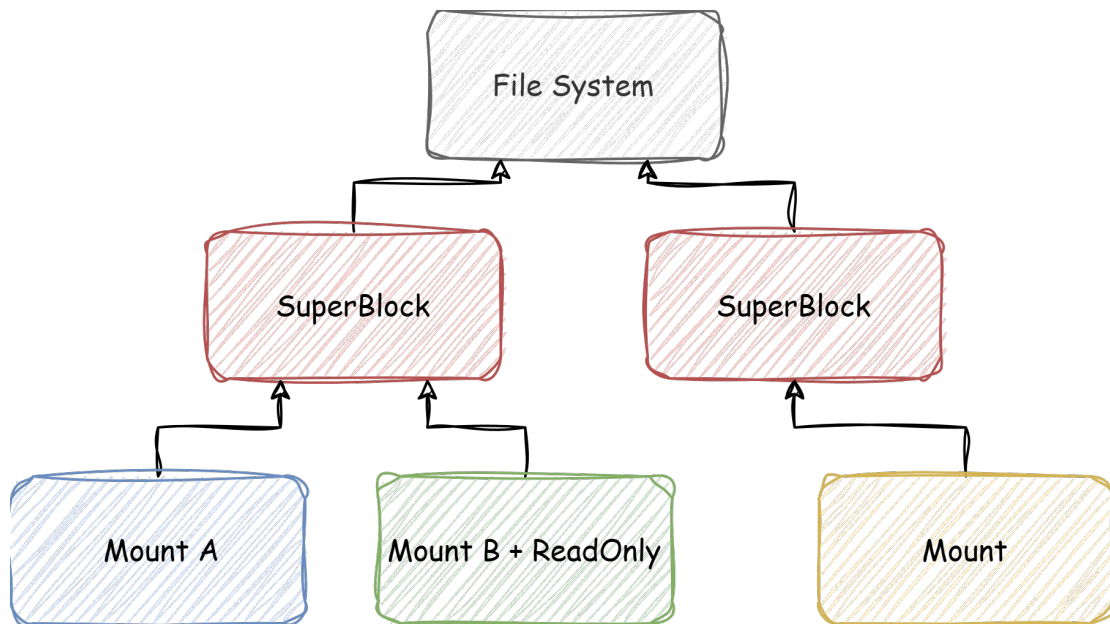


图 8 FileSystem, SuperBlock, Mount, 层层叠加的关系

6.1.4 Inode

Inode 是文件系统对象和元数据的核心抽象。VFS 在调用 `InodeOps` 前处理路径权限、挂载可写性和通用错误语义；后端则实现目录查找、创建、链接、删除、重命名、打开、截断、符号链接读取和属性查询等真正依赖文件系统格式的操作。

```
pub struct InodeOps {
    pub lookup: fn(&InodeRef, &str) → Result<InodeRef, SysError>,
    pub touch: fn(&InodeRef, &str, InodePerm) → Result<InodeRef, SysError>,
    pub mkdir: fn(&InodeRef, &str, InodePerm) → Result<InodeRef, SysError>,
    pub symlink: fn(&InodeRef, &str, &Path) → Result<InodeRef, SysError>,
    pub link: fn(&InodeRef, &str, &InodeRef) → Result<(), SysError>,
    pub unlink: fn(&InodeRef, &str) → Result<(), SysError>,
    pub rmdir: fn(&InodeRef, &str) → Result<(), SysError>,
    pub rename:
        fn(&InodeRef, &str, &InodeRef, &str, RenameFlags) → Result<(), SysError>,
    pub open: fn(&InodeRef) → Result<OpenedFile, SysError>,
    pub truncate: fn(&InodeRef, u64) → Result<(), SysError>,
    pub read_link: fn(&InodeRef) → Result<PathBuf, SysError>,
    pub get_attr: fn(&InodeRef) → Result<InodeStat, SysError>,
}
```

代码 18 InodeOps 覆盖目录、文件、链接和元数据操作

我们还在 SuperBlock 中维护了一个 icache, 这样, 已经被加载的 Inode 可以缓存在内存中, 如果后续再次命中, 就可以大大提高系统效率。而同时, 我们还引入了 Inode Shinker 内核线程。当内核内存占用过多时, shrinker 会遍历各个超级块, 尝试驱逐引用计数已经归零的 Inode。

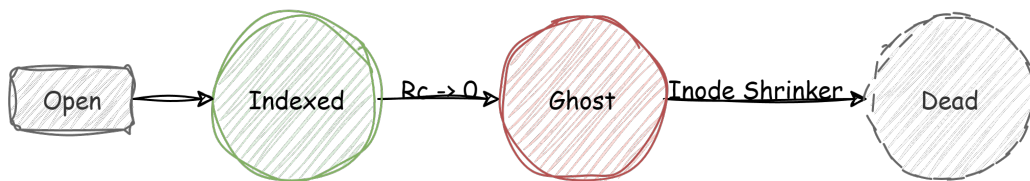


图 9 一个 Inode 的生命周期

6.1.5 Dentry

Dentry 保存“名字到 inode”的路径结构。目录 dentry 会维护子项弱引用缓存, 路径查找先尝试命中已有 dentry, 未命中时再通过父目录的 `InodeOps::lookup` 向

后端请求 inode 并创建新的 dentry。它不记录 mount 关系；跨挂载点的位置被 `PathRef { mount, dentry }` 共同表达。

```
pub struct Dentry {
    parent: Option<Arc<Dentry>>,
    inode: InodeRef,
    inner: RwLock<DentryInner>,
}

struct DentryInner {
    name: String,
    children: Option<HashMap<String, Weak<Dentry>>>>,
}
```

代码 19 Dentry 保存路径树节点和目录子项缓存

进入决赛后, Cargo 和编译器带来了数量巨大的重复路径查询。我们为已经成功解析的目录项增加了有界驻留: 最近使用的 positive dentry 可以继续由 VFS 复用, 而不必每次都进入 Ext4 查找同一个名字。缓存仍然只是路径解析的加速层, 文件身份与内容继续由 inode 和后端文件系统维护。这项改进既缩短了复杂构建中的公共路径, 也让 Dentry 从一个结构设计真正成长为能够服务大型工作负载的缓存机制。

6.1.6 File

`File` 表示一次打开后的文件对象, 它保存打开时的 `PathRef`、文件操作表、文件模式、后端私有数据和文件光标。普通文件读写使用 VFS 管理的光标; Anemone 还支持 stream 类对象, 允许后端文件系统可以在打开时标记自己的文件模式, 让后端自行维护读写位置。

```

pub struct FileOps {
    pub read:
        fn(&File, pos: &mut usize, buf: &mut [u8], FileIoCtx)
        → Result<usize, SysError>,
    pub write:
        fn(&File, pos: &mut usize, buf: &[u8], FileIoCtx)
        → Result<usize, SysError>,
    pub read_at:
        fn(&File, pos: usize, buf: &mut [u8], FileIoCtx)
        → Result<usize, SysError>,
    pub write_at:
        fn(&File, pos: usize, buf: &[u8], FileIoCtx)
        → Result<usize, SysError>,
    pub check_status_flags:
        fn(&File, FileOpStatusFlags) → Result<(), SysError>,
    pub seek: fn(&File, pos: &mut usize, SeekFrom) → Result<usize,
SysError>,
    pub read_dir:
        fn(&File, pos: &mut usize, &mut dyn DirSink)
        → Result<ReadDirResult, SysError>,
    pub poll:
        for<'a> fn(&File, &PollRequest<'a>)
        → Result<PollRegisterResult, SysError>,
    pub fcntl: Option<FileFcntlHook>,
    pub ioctl: for<'a> fn(&File, IoctlCtx<'a>) → Result<u64,
SysError>,
}

```

代码 20 FileOps 是打开文件对象的统一操作表

我们还把文件描述符表中的 descriptor 与真正的 opened file description 明确区分。dup 与 fork 可以让多个 fd 共享同一个打开文件对象，因此文件偏移、状态标志和 POSIX record lock 等行为必须跟随正确的生命周期。这个区分支撑了 flock、POSIX byte-range lock、close-on-exec、fd duplication 和 Unix Socket 等决赛阶段能力，也让用户程序看到的文件语义更接近完整的 Unix 环境。

6.2 挂载树

Anemone 用全局 VFS 子系统维护可见挂载树和匿名挂载树。可见挂载树承载根文件系统、用户可见的磁盘文件系统、ramfs、procfs 和 devfs；匿名挂载树用于 pipe、

eventfd、timerfd 等不需要路径查找的内核内部文件对象。路径查找后, 得到 `PathRef`, 其中同时包含当前挂载点和 dentry, 因此同一个 inode 经由不同 bind mount 被访问时仍能保留路径视图差异。

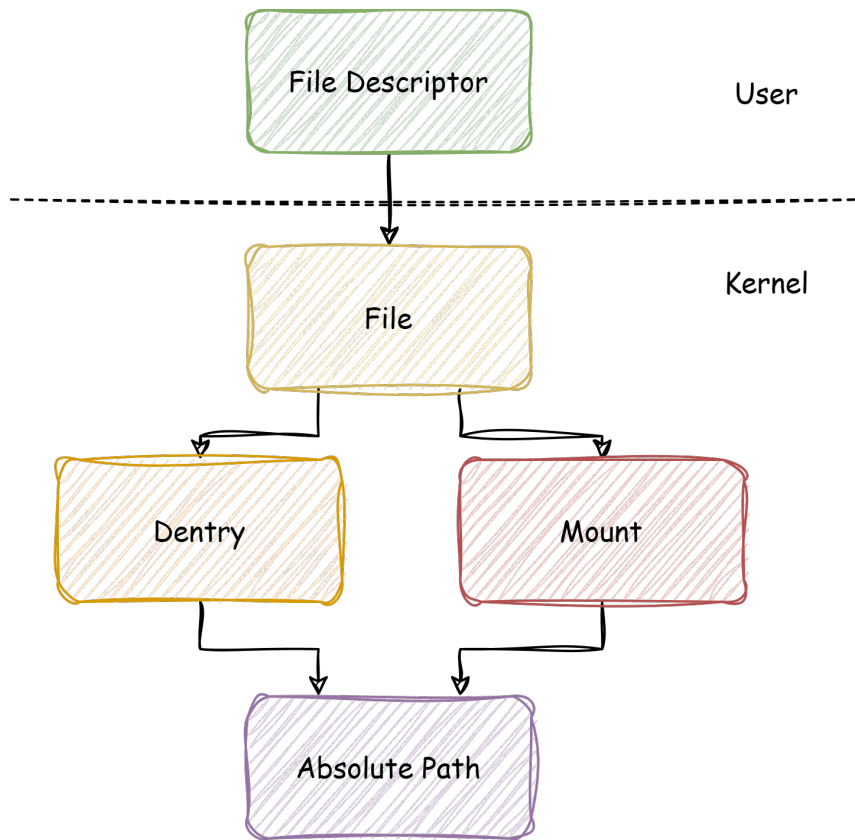


图 10 路径查询的例子

挂载操作集中由 `MountTree` 处理。它支持根挂载、普通挂载、bind / recursive bind、move mount、remount 属性更新、private propagation 请求、普通卸载和 lazy detach。挂载树是 mount 位置的唯一写入者, `Dentry` 不反向记录挂载关系; 路径查找遇到挂载点时查询当前 mount stack 的最上层视图。如果查找过程中挂载树发生变化, VFS 通过 placement generation 重试, 避免返回过期路径视图。这样, 我们就避免用户看到过期状态。

6.3 VMO 形式的页缓存

普通文件的缓存直接复用内存管理章节介绍过的 VMO 机制。每个 regular inode 拥有一份 address space, 普通 read / write 通过它复制数据, 文件 mmap 的缺页路径也从同一批缓存页取得物理内存。Ext4 与 ramfs 只需要实现如何填充和写回页面, 不需要各自维护一套 mmap 缓存。

举个例子。读路径第一次访问某页时, 从硬盘读取文件内容并填入新分配的 frame; 后续读或映射访问就能直接命中缓存。而写路径会在必要时先加载旧页, 修改 frame 后把页标记为 dirty。sync_range、sync_all 和文件系统同步路径再把脏页写回后端。文

件截断时, Anemone 会使可见范围发生变化的缓存页失效, 下一次访问再从后端重新载入, 避免旧页内容跨越新的文件大小边界。

针对编译场景中连续读取源码、元数据与中间产物的特点, 我们又加入了有界批量 I/O。连续缺失的缓存页可以一次从 Ext4 填充, 连续脏页也可以成批写回, 减少反复进入文件系统和块设备的开销。用户缓冲区的读写则可以通过专门的 `direct-user` 路径按页推进, 保留 Linux 风格的短读、短写和部分完成语义。

6.4 Ext4 支持

Anemone 通过经典的 `lwext4` C 库接入 Ext4, 但我们没有把 C 接口直接暴露给整个内核, 而是在 Rust 侧建立安全封装, 再由 VFS 与 `address space` 使用。文件与目录操作由可睡眠的文件系统互斥机制串行化, 使一次 `rename`、`unlink`、`create` 或 `metadata` 更新不会被拆成彼此交错的零散调用。

在决赛阶段, 我们继续补齐了 `umask` 与节点创建、`rename`、`named FIFO`、`flock`、`POSIX record lock`、同步与页缓存等能力。最终, Ext4 不再只是“能够挂载和读写文件”的后端, 而是可以支撑 `shell`、`glibc`、`Cargo`、`rustc` 和链接器长期协同工作的主文件系统。

7 设备驱动模型

Anemone 的设备驱动模型参考了 Linux 的 device / driver / bus 分层，但没有把设备发现、驱动绑定和用户态文件接口混在 VFS 内部。设备模型负责发现硬件、匹配驱动、保存驱动私有状态；VFS 只处理路径、inode 和 opened file object；`devfs` 作为中间桥，把已经注册的字符设备和块设备发布为 `/dev` 下的节点。这样，设备驱动模型与 VFS 对彼此的内部状态保持解耦，设备语义仍由对应的 driver owner 决定。

许多 I/O 操作会同时经过路径查找、设备节点、file operation、ioctl 和后端驱动。如果这些职责直接堆在 open 或 ioctl 的 syscall 层，增加块设备、串口、随机数设备、SD 卡和 PCIe 设备时很容易变成特判集合，或者大幅污染内核代码的边界，导致后续维护很困难。

为了解决这种问题，我们把“设备如何被发现和绑定”与“设备如何以文件形式暴露”分开，使同一套驱动模型可以服务 RISC-V、LoongArch、VirtIO MMIO、VirtIO PCIe 等不同平台和传输方式。

7.1 核心对象

设备驱动模型的核心对象是 `BusType`、`Device` 和 `Driver`。`BusType` 保存本总线上的设备和驱动集合，并定义匹配规则；`Device` 表示一个已经发现的设备节点，保存父子拓扑、固件节点和 driver 私有状态；`Driver` 提供 probe、shutdown 等操作，并记录自己已经绑定的设备。

```

pub trait BusType: KObject {
    fn base(&self) → &BusTypeBase;
    fn matches(&self, device: &dyn Device, driver: &dyn Driver) →
bool;
    fn register_device(&self, device: Arc<dyn Device>);
    fn register_driver(&self, driver: Arc<dyn Driver>);
}

pub trait DeviceData: KObject {
    fn base(&self) → &DeviceBase;
}

pub trait DeviceOps {}
pub trait Device: DeviceData + DeviceOps {
    fn driver(&self) → Option<Arc<dyn Driver>>;
    fn set_driver(&self, driver: Option<Arc<dyn Driver>>);
    fn set_drv_state(&self, state: AnyOpaque);
    fn add_child(&self, child: Arc<dyn Device>);
    fn fwnode(&self) → Option<&Arc<dyn FwNode>>;
}

pub trait DriverData: KObject {
    fn base(&self) → &DriverBase;
}

pub trait DriverOps {
    fn probe(&self, device: Arc<dyn Device>) → Result<(),
SysError>;
    fn shutdown(&self, device: &dyn Device);
    fn as_platform_driver(&self) → Option<&dyn PlatformDriver>;
    fn as_virtio_driver(&self) → Option<&dyn VirtIODriver>;
    fn as_pcie_driver(&self) → Option<&dyn PcieDriver>;
}

pub trait Driver: DriverData + DriverOps {
    fn attach_device(&self, device: Arc<dyn Device>);
}

```

代码 21 `BusType`、`Device` 和 `Driver` 是设备发现、匹配和绑定的核心接口。这三个对象的关系决定了 Anemone 的 probe 流程。内核启动时，会先注册内置驱动，随后解析固件描述并创建设备对象；总线在注册设备或注册驱动时都会尝试匹配未绑

定对象。匹配成功后，driver 的 `probe` 初始化具体硬件或传输层，成功后设备记录对应 driver，driver 也记录自己已经管理的设备。

`DeviceBase` 中的 `drv_state` 是这里的关键点。设备模型只负责保存一个类型擦除的私有状态槽，不解释状态内容；具体 driver 在 `probe` 成功后写入自己的状态对象，后续 shutdown、interrupt handler 或设备文件操作再按真实类型取回。这样，通用设备框架不需要知道 VirtIO block、串口、PCIe transport 等具体实现的数据结构，驱动私有状态也不会反向污染 VFS 或 syscall 层。

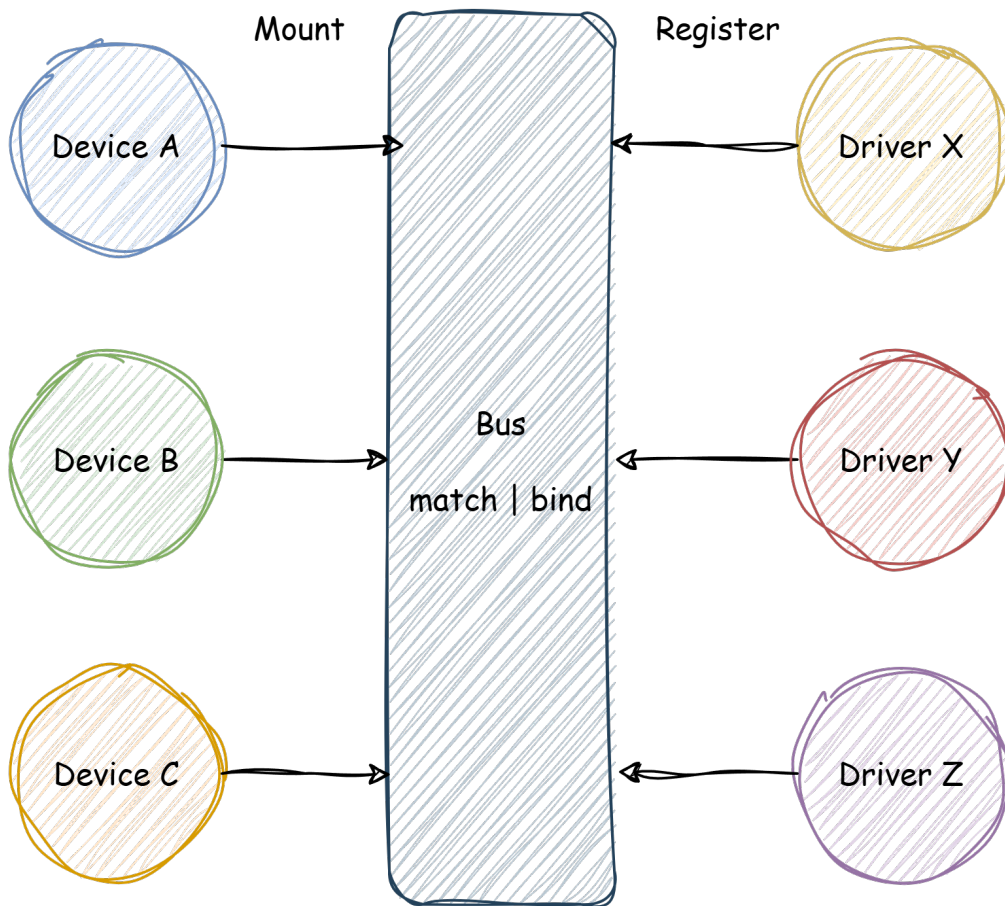


图 11 设备，驱动通过总线相联结

通过过这种机制，我们就漂亮地解耦了 VFS 和设备驱动系统。

7.1.1 Bus

Anemone 目前已经支持三类总线：Platform、VirtIO 和 PCIe。它们共用 `BusType` 的设备 / 驱动集合与 `probe` 流程，但匹配规则不同。

Platform bus 面向设备树发现出来的 MMIO 设备。`PlatformDevice` 持有固件节点和 MMIO resource，`PlatformDriver` 暴露一组 compatible 字符串，总线通过设

备树中的 `compatible` 属性选择 driver。这让串口、VirtIO MMIO transport、RTC、platform interrupt controller 等设备可以通过同一类固件描述进入设备模型。

VirtIO bus 位于 transport 之后。无论底层 transport 来自 MMIO 还是 PCIe, transport driver probe 成功后都会创建一个 `VirtIODevice`, 再把它注册到 VirtIO bus。VirtIO driver 不关心 transport 的来源, 只根据 VirtIO device type 匹配。例如 VirtIO block driver 只需要声明自己支持 block device type, 实际设备可以来自 QEMU virt 的 MMIO, 也可以来自 PCIe 枚举。

PCIe bus 的匹配和初始化更复杂。它既可以按 vendor / device id 匹配, 也可以按 class code 匹配; probe 之前还要完成 BAR 探测、aperture 分配和 MMIO remap。Anemone 把这些资源分配放在 PCIe bus / PCIe driver 接口内完成, 所有复杂度都不泄露出去。

7.1.2 Device

`Device` 是设备模型中的拓扑节点。设备对象可以有父子关系, 例如 platform 设备下面生成 VirtIO MMIO transport, transport 再生成具体 `VirtIODevice`; PCIe host bridge 枚举出 endpoint, VirtIO PCIe transport 又把 endpoint 转换成 VirtIO 设备。这个拓扑让 shutdown 可以按深度优先顺序通知子设备先退出, 再处理父设备, 避免父 transport 先消失后子设备还尝试访问硬件。我们在关机之前, 就会先沿着设备树拓扑逐个关闭设备, 正确的顺序确保了我们不会出现“先关闭父设备, 再关闭连着父设备的子设备”的错误行为。

设备对象还连接固件描述和 driver 私有状态。Platform device 从设备树节点读取 `compatible`、`reg`、`interrupt-parent` 等属性; PCIe device 保存配置空间、BAR 和 domain 资源; VirtIO device 保存 transport。通用 `Device` trait 只暴露少量能力: 取回当前绑定的 driver、记录 driver state、增加子设备和访问固件节点。这样的接口足够支撑通用 probe 流程, 但不会让下层驱动依赖不属于自己的内部状态。

7.1.3 Driver

`Driver` 的核心入口是 `probe`。`probe` 成功意味着 driver 已经接管该设备, 并且可以把后续需要的状态写入 `drv_state`。

不同总线通过 `as_platform_driver`、`as_virtio_driver`、`as_pcie_driver` 取得自己的窄接口。Platform driver 暴露 match table, VirtIO driver 暴露 device id table, PCIe driver 暴露 class code / vendor-device table 和资源初始化 hook。这种区分让通用 `DriverOps` 不需要知道每种总线的匹配细节, 总线也不需要猜测某个 driver 的私有类型。

7.2 字符设备与块设备

设备被 driver 接管后，还需要以用户态可访问的形式暴露出来。Anemone 把这一步放在字符设备和块设备子系统中完成。字符设备提供字节流语义，块设备提供固定块读写语义；二者都先注册到自己的 subsystem，再由 subsystem 发布到 devfs。

```
pub trait CharDev: Send + Sync {
    fn devnum(&self) → CharDevNum;
    fn read(&self, buf: &mut [u8]) → Result<usize, SysError>;
    fn write(&self, buf: &[u8]) → Result<usize, SysError>;
    fn seek(&self, ctx: CharSeekCtx<'_>) → Result<usize, SysError>;
    fn ioctl(&self, ctx: CharIoctlCtx<'_>) → Result<u64, SysError>;
}

pub trait BlockDev: Send + Sync {
    fn devnum(&self) → BlockDevNum;
    fn block_size(&self) → BlockSize;
    fn total_blocks(&self) → usize;
    fn read_blocks(&self, block_idx: usize, buf: &mut [u8]) → Result<(), SysError>;
    fn write_blocks(&self, block_idx: usize, buf: &[u8]) → Result<(), SysError>;
    fn ioctl(&self, ctx: BlockIoctlCtx<'_>) → Result<u64, SysError>;
}
```

代码 22 CharDev 和 BlockDev 分别定义字符设备与块设备的最小能力接口。字符设备侧，比如 /dev/null、/dev/zero、/dev/full 和 /dev/urandom 都通过同一类 CharDev 接口暴露读写语义。统一的 char devfs file ops 只负责从 inode 的 rdev 找到对应设备，再把 read、write、seek 和 ioctl 分发给 CharDev。这样，内存类字符设备的 seek 行为和随机数设备的读取行为都留在具体设备内部，而不是写成 devfs 的设备号特判。交互式串口则在决赛阶段由专门的 TTY 子系统接管。

块设备侧，后端 BlockDev 只承诺按 block size 对齐的块读写。用户态通过 /dev/vda、/dev/loop0 等块特殊文件访问时，devfs 的 block file ops 在前端提供封装后的、字节级读写（这一点也是为了对齐 Linux）和 seek：对非对齐读写使用中间缓冲做 read-modify-write，对 BLKGETSIZE64、BLKSSZGET 等通用 BLK* ioctl 先在 block

subsystem 内处理, 剩余私有命令再交给具体 `BlockDev::ioctl`。这使后端驱动可以保持清晰的块设备契约, 同时让 Linux 用户态看到接近普通块特殊文件的字节接口。

7.3 TTY 与交互式终端

初期的串口只承担控制台输出和非常简单的输入, 足以观察启动过程, 却不能支撑真正的交互式 shell。决赛阶段, 我们在 UART 之上建立了独立的 TTY 子系统: 串口驱动只负责收发原始字节, Terminal 对象处理 canonical/raw 输入、回显、控制字符、输出转换、termios、窗口大小和 poll/epoll readiness。

TTY 会把稳定的 `/dev/ttyS0` 发布到 devfs, 并让 `/dev/tty` 根据调用进程返回其 controlling terminal。结合进程管理章节介绍的 session、process group 和作业控制, shell 可以切换前台作业, Ctrl-C 与 Ctrl-Z 可以发送给正确的进程组, BusyBox vi 等全屏程序也可以通过 termios 使用终端。我们没有为 shell 或某个应用编写输入特判, 而是形成了一条从 UART 中断、TTY 行规、文件对象到进程组信号的完整路径。

7.4 设备树与平台初始化

Anemone 的平台发现以设备树为主要入口。启动早期, 内核从固件传入的 FDT 中扫描内存、CPU 数量和时钟频率等必须在完整初始化前知道的信息; 进入通用初始化后, 内核把 FDT unflatten 成长期驻留的设备树对象, 并通过路径、phandle、compatible、reg、ranges 等接口服务后续平台发现。

我们实现了自己的 `device-tree crate` 来保存 unflatten 之后的树结构, 它已经通过了 Miri 的验证。DeviceTreeHandle 持有 arena-backed 的不可变树, DeviceNodeHandle 可以稳定引用其中节点; 平台发现逻辑据此遍历 simple-bus 下的子节点, 为带 compatible 且 status 可用的节点创建 PlatformDevice, 解析 MMIO resource, 并注册到 Platform bus。进入这一步之后, 设备发现就不再依赖架构启动代码里的硬编码分支, 而是回到统一的 bus matching 和 driver probe。

在设备树之外, Anemone 还保留了一层很薄的 MachineDesc。它用来根中断控制器、早期 timer 这类必须在完整驱动设备初始化前完成的初始化。

```
pub trait MachineDesc: Sync {
    fn compatible(&self) → &[&str];
    unsafe fn early_init_intc(&self);
    unsafe fn early_init_timer(&self);
}
```

代码 23 MachineDesc 只覆盖完整设备驱动模型建立前必须完成的板级早期入口

这个设计参考了 Linux ARM 的 `machine_desc` / `DT_MACHINE_START` 思路。后来的通用架构越来越多把平台初始化推向设备树驱动的统一发现路径，但我们没有完全走纯设备树路线：Anemone 当前需要支持 RISC-V 和 LoongArch 两套启动入口，根中断控制器和早期 timer 又必须先于普通 platform probe 建立。我们把这些少量板级差异压缩到 `MachineDesc` 中，再把常规设备交给设备树和 Platform bus。这样既保留了设备树的可扩展性，也避免为了少数早期入口把通用设备模型设计得过重。

相比不少往届作品中常见的板级硬编码初始化，这套抽象让迁移新平台的改动更集中。新的平台通常需要补充机器描述、设备树节点和少量 driver match table，而不是在主初始化流程里散布新的条件分支。对 Anemone 来说，这正好满足当前阶段的需求：复杂度可控，同时具备向更多 QEMU machine 或真实开发板迁移的空间。

7.5 从虚拟机到真实开发板

决赛阶段，设备模型开始真正服务多种机器，而不再只围绕 QEMU。RISC-V 一侧加入了 VisionFive 2 所需的 PLIC、DesignWare MSHC 和 SD Memory 路径；LoongArch 一侧适配了 Loongson 2K1000、LS7A RTC、中断控制器和板级设备。VirtIO MMIO、VirtIO PCIe、SD 卡与其它块设备最终都接入同一套 block/VFS 接口，上层文件系统不需要知道启动盘来自哪一种控制器。

为了管理这些组合，我们还把 platform、system target、kernel configuration、rootfs 与 QEMU invocation 分开描述。同一份内核机制可以面向虚拟机、竞赛环境或真实开发板选择不同的设备树、启动程序和根文件系统，而不必在内核源码中加入“决赛模式”一类分支。这让双架构和多平台从代码目录上的支持，进一步变成了可以重复构建和启动的系统产品。

7.6 Devfs 桥接

`devfs` 是设备模型与 VFS 之间的桥。设备子系统完成注册后，可以可选地把设备发布到设备文件系统；`devfs` 保存稳定的名字、inode 编号、权限和 `rdev`，但不拥有具体设备语义。打开设备节点时，`devfs` 调用发布记录中的 `DevfsNodeOps::open()`，由字符或块设备子系统返回真正的文件操作表。

```
pub trait DevfsNodeOps: Send + Sync {  
    fn open(&self, inode: &InodeRef) → Result<OpenedFile,  
        SysError>;  
    fn get_attr(&self, inode: &InodeRef, attr: DevfsNodeAttr)  
        → Result<InodeStat, SysError>;  
}
```

代码 24 `DevfsNodeOps` 只负责把 devfs inode 接回设备子系统拥有的 file behavior 简单来说——driver probe 产生设备后端，char / block subsystem 注册设备号和名字，devfs 发布 inode，VFS open 得到设备子系统提供的 `FileOps`。之后 read、write、seek、ioctl 都沿着 `FileOps` 回到对应的 `CharDev` 或 `BlockDev`。因此，`/dev` 看起来是文件系统命名空间的一部分，但设备行为仍然由设备 owner 决定。

这种设计就把文件系统和设备驱动漂亮地解了耦。VFS 负责路径、挂载、inode 和 opened file description；设备驱动模型负责硬件发现、驱动绑定和设备能力；`devfs` 只承担发布和分发，不把 VFS 变成驱动框架，也不把驱动框架变成路径查找系统。决赛阶段的 devfs 还支持层级目录发布，使设备可以按稳定结构出现在 `/dev` 下，而不必把所有节点都挤在根目录。对于后续扩展 sysfs、更多字符设备、更多块设备或热插拔协议，这个边界也为我们保留了继续演进的空间。

8 网络栈

初赛结束时，网络仍然是 Anemone 计划中的下一项能力。进入决赛阶段后，我们完成了从网络设备驱动、IPv4 控制面、协议栈，到 Socket 和只读 Netlink 诊断面的整条路径。现在，Anemone 已经可以运行 `ping`，执行 `git clone`，使用包管理器完成 `apt install`，也可以直接运行 `ip link/address/route show` 和 `ss -tan` 等工具查看系统中的链路、地址与路由，我们的网络能力已经进入了可用状态。

8.1 Socket 能力全景

Anemone 的通用 Socket 前端负责接收系统调用、处理用户内存与 Linux ABI，再把操作分派给具体协议族。前端使用一组静态操作表描述每种 Socket 的实际能力。数据面按照字节流 (byte stream)、数据报 (datagram) 和有边界的可靠报文 (seqpacket) 三种形状组织，其余操作则按协议族是否真正支持来安装。

```
pub(super) enum SocketIoOps {
    ByteStream {
        socket_type: SocketType,
        send: SocketSendOp,
        receive: SocketReceiveOp,
    },
    Datagram {
        socket_type: SocketType,
        send: SocketSendOp,
        receive: SocketReceiveOp,
    },
    Seqpacket {
        socket_type: SocketType,
        send: SocketSendOp,
        send_wait: SocketSendWaitOp,
        receive: SocketReceiveOp,
    },
}
```

代码 25 SocketIoOps 区分 stream、datagram 与 seqpacket 三种数据面能力 (节选)


```
pub(super) struct SocketOps {
    pub io: SocketIoOps,
    pub create: Option<fn() → Result<SocketPreparation,
SysError>>,
    pub create_pair: Option<fn() → Result<SocketPairPreparation,
SysError>>,
    pub bind:
        Option<fn(&AnyOpaque, SocketAddress) → Result<(),
SocketBindError>>,
    pub listen:
        Option<fn(&AnyOpaque, i32) → Result<(),
SocketListenError>>,
    pub connect:
        Option<fn(&AnyOpaque, SocketAddress) → Result<(),
SocketConnectError>>,
    pub accept:
        Option<fn(&AnyOpaque) → Result<SocketAcceptItem,
SocketAcceptError>>,
    pub shutdown:
        Option<fn(&AnyOpaque, SocketShutdown) → Result<(),
SocketShutdownError>>,
    /* 地址查询、option、poll 与 final release 见下段。 */
}
```

代码 26 SocketOps 静态装配创建、连接与连接生命周期操作（节选）


```
// SocketOps 的查询、option、poll 与 release 字段 (续)
pub local_address:
    Option<fn(&AnyOpaque, &mut dyn SocketAddressSink)
        → Result<(), SocketQueryError>>,
pub peer_address:
    Option<fn(&AnyOpaque, &mut dyn SocketAddressSink)
        → Result<(), SocketQueryError>>,
pub accepting:
    fn(&AnyOpaque) → Result<bool, SocketQueryError>,
pub query_option:
    Option<fn(&AnyOpaque, SocketOptionQuery)
        → Result<SocketOptionValue, SocketOptionError>>,
pub mutate_option:
    Option<fn(&AnyOpaque, SocketOptionMutation)
        → Result<(), SocketOptionError>>,
pub poll:
    for<'a> fn(&AnyOpaque, &PollRequest<'a>)
        → Result<PollRegisterResult, SysError>,
pub final_release:
    fn(&AnyOpaque, SocketReleaseReason),
```

代码 27 SocketOps 继续装配查询、option、readiness 与最终释放能力 (节选, 续)

我们用这套虚表同时承载 IPv4 TCP、UDP、ICMP raw、Unix stream/seqpacket 和 Netlink 等协议族, 将这些不同的后端能力统一为表现为文件的 Socket。各协议族继续拥有自己的连接、报文、命名空间和错误语义。这样, 一条系统调用可以复用公共的 Linux ABI 与用户内存处理, 而不会迫使 TCP、UDP、Unix Socket 和 Netlink 共享一份含义模糊的内部状态。

8.2 从调研到四层架构

在设计网络栈之前, 我们广泛调研了往届参赛内核。几乎所有实现都选择把 smoltcp (一个广泛使用的嵌入式网络协议栈 crate) 的对象与语义直接嵌入内核: Socket 对象保存 smoltcp 对象句柄, 系统调用路径读取 smoltcp 状态, 并据此计算 Linux 可见的 readiness 和错误。这条路线可以很快建立基本连通性, 代码量也较少; 但是, 随着 bind、listen、accept、非阻塞 I/O、poll/epoll、close 和诊断能力逐步增加, 具体后端的对象模型显然会扩散到越来越多的内核模块。

所以我们的选择是多建立一道边界。smoltcp 在 Anemone 中是协议引擎后端, 它的对象句柄 (handle)、Socket 集合、缓冲区和状态表示全部留在协议栈实现内部。内核通

过规范化语义访问协议能力, 并自行拥有 Linux Socket 的 fd、等待、readiness、errno 和生命周期。这项选择增加了早期设计与适配工作, 也换来了清晰得多的对象关系。

我们把唯一负责保存和改变某项状态的模块称为状态属主(owner)。在此基础上, 我们把跨模块的对象隔离原则称为 **object fence**: 属主的内部对象停留在自己的边界内, 跨层只传递规范化的值、不透明标识、一次性事实快照、重新检查通知或受限操作能力。接收方可以请求操作、观察事实和重新检查, 却不能沿着一个对象句柄穿透到另一层的私有状态机。

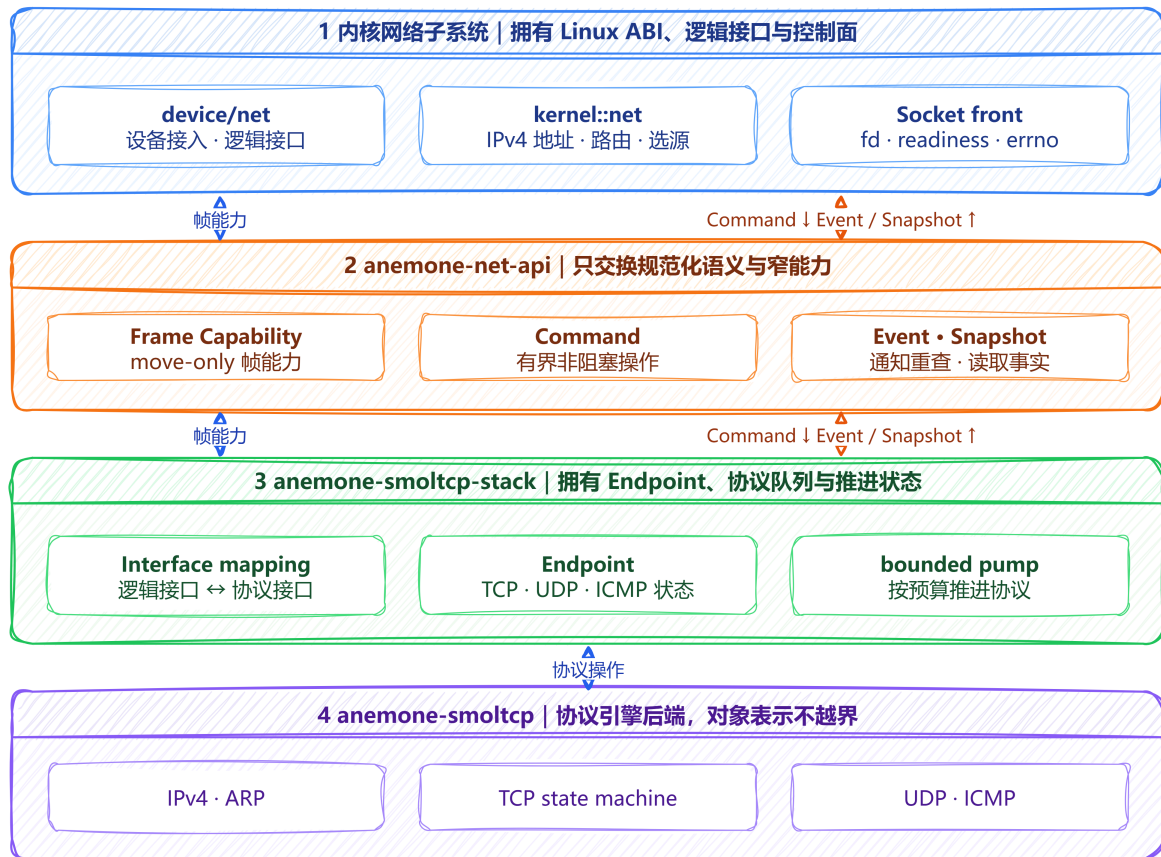


图 12 Anemone 网络栈的四层架构。数据包、操作命令、要求重新读取事实的事件, 以及读取时刻的事实快照通过不同边界传递, 各层只拥有自己的对象与状态。

四层分别承担以下职责:

表 2 四层网络架构的状态所有权

层次	实现	职责与 object fence
内核内部网络子系统	<code>device/net</code> 、 <code>kernel::net</code> 、Socket 前端	拥有设备接入、内核逻辑网络接口、IPv4 路由与源地址选择、fd、等待和 Linux ABI；只使用规范化协议事实，不接触 smoltcp 对象句柄。
Net API	<code>anemone-net-api</code>	定义帧交接、命令、重新检查通知、事实快照和受限操作能力；类型中不出现内核对象、smoltcp 对象或网卡 DMA/队列描述符编号。
协议栈	<code>anemone-smoltcp-stack</code>	拥有协议接口映射、网络端点 (Endpoint)、TCP/UDP/ICMP 状态与协议队列，并接单轮预算推进；不持有 fd、任务、内核等待关系或 Linux errno。
协议栈后端	<code>anemone-smoltcp</code>	执行具体 TCP/IP 协议机制；其网络接口、Socket 集合、对象句柄与缓冲区表示不越过协议栈层。

这四层并非简单的逐层转发关系。`anemone-net-api` 是内核和协议栈共同依赖的语义接口；`anemone-smoltcp-stack` 同时依赖 Net API 和协议引擎后端；smoltcp 本身不依赖任何 Anemone 模块。编译依赖的方向保证底层无法看到内核对象，Rust 的可见性规则与受限操作类型继续约束运行时能够交换的内容。

8.2.1 Command、Event 与 Snapshot

Net API 的上边界围绕三类交互组织。它们把“请求协议操作”“得知事实可能改变”和“读取当前事实”分开，使通知不会悄悄变成第二份协议状态。

表 3 Net API 的 Command、Event 与 Snapshot 三类核心交互

交互语义	当前代码中的形态	边界含义
Command (命令)	<code>create</code> 、 <code>bind</code> 、 <code>connect</code> 、 <code>send</code> 、 <code>receive</code> 、 <code>release</code> 等方法	请求一次有界、非阻塞的协议操作； 若产生待发送工作, 则通知网络工作 线程继续处理, 不持有任务, 也不在 内部等待。
Event (事 件)	TCP、UDP、ICMP 各 自的端点失效通知	只表示“相关事实可能变化, 请重新 读取”; 可以合并, 不携带 readiness 位图、错误码或连接阶段。
Snapshot (快照)	各协议的端点事实、接 口事实与诊断记录	给出读取时刻的一次性事实; 内核据 此计算 readiness、errno 和诊断输 出, 收到事件后重新取得快照。

这三者是我们的核心设计模型。不过, 源码没有建立一个包罗所有协议的 `Command`、`Event` 或 `Snapshot` 总枚举。代码形状上, 三类语义分别为表现为窄方法和 TCP、UDP、ICMP 各自的具体类型, 新增协议无需不断扩大一组公共大枚举。它们共享边界规则: 命令不阻塞, 事件只要求重新检查, 快照只描述读取时刻的事实。所有这些能力都是非阻塞的, 协议栈不负责同步能力, 这些东西归内核内部处理。这样, 我们能通过非阻塞的原语自由组合, 配上内核的等待原语, 实现各种阻塞和非阻塞的 Linux ABI。

8.3 帧路径: 资源身份停留在设备侧

我们用协议栈和网卡驱动的下边界作为一个例子讲讲 Object fence 这个设计原则。简单来说, 网卡驱动拥有收发帧的底层存储、队列描述符、DMA 映射、可用队列槽数量和完成回收状态等设备内部状态。协议栈只需要“取得一帧并消费”或“取得一个发送槽并填充”的能力。我们使用只能移动一次的 token 表达这次临时所有权, 帧内存的借用期被限制在回调函数内, 协议栈无法把设备侧存储保存到自己的对象中。

```
pub trait RxToken {
    fn consume<R, F>(self, f: F) → R
    where
        F: FnOnce(&[u8]) → R;
}

pub trait TxToken {
    fn capacity(&self) → usize;

    fn consume<R, F>(
        self,
        len: usize,
        f: F,
    ) → Result<R, FrameSizeError>
    where
        F: FnOnce(&mut [u8]) → R;
}

pub enum ReceiveOutcome<R, T> {
    Ready { rx: R, tx: T },
    Empty,
    TransmitExhausted,
    LinkUnavailable,
}

pub enum TransmitOutcome<T> {
    Ready(T),
    Exhausted,
    LinkUnavailable,
}
```

代码 28 一次性 RX/TX token 与明确的结果枚举共同描述帧资源交接（节选）

```
pub trait FrameProvider {
    type RxToken<'a>: RxToken where Self: 'a;
    type TxToken<'a>: TxToken where Self: 'a;

    fn receive(&mut self, now: Instant)
        → ReceiveOutcome<Self::RxToken<'_>, Self::TxToken<'_>>;
    fn transmit(&mut self, now: Instant)
        → TransmitOutcome<Self::TxToken<'_>>;
    fn capabilities(&self) → FrameCapabilities;
    fn link_state(&self) → LinkState;
}
```

代码 29 `FrameProvider` 只向协议栈开放取得帧、发送槽和链路事实的受限接口（节选，续）

`ReceiveOutcome` 把接收帧与一个可能用于立即回复的发送槽一起交给协议引擎，这是因为 `smoltcp` 处理一份输入时可能立刻产生响应。设备队列为空、发送槽耗尽和链路不可用都是正常结果；协议栈不会为这些情况触发内核异常，也不会把它们折叠成同一个模糊的失败。

协议推进由 `Stack::pump` 完成；这里的 `pump` 指在给定预算内让协议状态机向前运行一轮。`&mut Stack` 是唯一允许推进协议状态的访问能力，调用方选择具体的帧提供者（`provider`）、接口、单调时间和单轮预算。返回值只描述是否仍有协议工作、是否可以立即再运行一轮，以及下一次协议定时期限。

```
pub struct PumpOutcome {
    pub work_remaining: bool,
    pub recheck: Recheck,
    pub next_deadline: Option<Instant>,
}

impl Stack {
    pub fn pump<P: FrameProvider>(
        &mut self,
        id: InterfaceId,
        provider: &mut P,
        now: Instant,
        budget: PumpBudget,
    ) → Result<PumpOutcome, PumpError> {
        /* 在有限收发预算内推进真实协议引擎;
           根据设备事件、剩余工作与定时期限返回 PumpOutcome。 */
    }
}
```

代码 30 `Stack::pump` 用泛型帧能力和显式预算推进协议状态（实现节选）

在实际内核运行中，`device/net` 接收网卡设备发布的帧提供者，网络接入模块为它分配一个逻辑网络接口，并把它交给网络工作线程。硬中断只确认设备事件并发出一次重新检查通知；工作线程随后在普通内核线程上下文中独占访问协议栈，完成有预算上限的收发工作，然后释放锁。关机时，网络子系统先关闭新的协议工作入口，再请求工作线程停止，设备驱动继续负责最终关闭中断、设备队列和 DMA 资源。

8.4 网络域与 IPv4 控制面

网络域指共享一组网络接口、地址与路由策略和协议状态的隔离范围。Anemone 当前使用一个初始网络域，它组合了一份全局协议栈状态、回环接口和已经成功接入的外部逻辑网络接口。我们在这里刻意保留了三种标识：`NetdevId` 标识驱动发布的网络设备，`ifindex` 和接口名称标识用户可见的逻辑接口，`InterfaceId` 标识协议栈内部的接口映射。它们之间存在受控关联，却不能相互推导或替代。设备重新发布、协议映射回滚和用户可见 `ifindex` 因而不会意外共享一套生命周期。

IPv4 地址和路由策略由独立的 `Ipv4ControlPlane` 模块维护。它根据启动配置建立 `127.0.0.1/8`、外部地址、直连路由与可选默认路由，并在每次发送前选择源地址和协议接口。协议栈接收选定结果并执行操作，不从 `smoltcp` 内部路由表反推内核策略。当

前静态控制面已经足以支撑 QEMU 与比赛环境中的稳定网络部署，同时也为以后增加动态配置保留了清楚的状态归属。

8.5 上边界：协议事实如何成为 Linux readiness

每个 TCP 网络端点 (Endpoint) 的连接阶段、收发容量、对端关闭和待处理错误都由协议栈中的 TCP 模块维护。内核 Socket 适配层只取得一次事实快照，并在自己的边界内计算 Linux poll/epoll 事件。协议栈发出的 Event 只表示“事实可能改变，请重新读取”；它本身不携带 readiness 位图，也不会成为第二份 readiness 状态。


```

fn project_public(
    facts: TcpEndpointFacts,
    interests: PollEvent,
) → PollEvent {
    match facts {
        TcpEndpointFacts::Idle | TcpEndpointFacts::Bound => {
            PollEvent::HANG_UP | (PollEvent::WRITABLE & interests)
        }
        TcpEndpointFacts::Listener { has_pending_child } => {
            if has_pending_child &&
interests.contains(PollEvent::READABLE) {
                PollEvent::READABLE
            } else {
                PollEvent::empty()
            }
        }
        TcpEndpointFacts::Connection(connection) => {
            let mut events = PollEvent::empty();
            let pending_error = connection.has_pending_error();

            if pending_error {
                events |= PollEvent::ERROR;
            }
            if connection.is_terminal() {
                events |= PollEvent::HANG_UP;
            }
            /* READABLE / WRITABLE / READ_HANG_UP 投影见下段。 */
            events
        }
    }
}

```

代码 31 Socket 层按网络端点形态计算 Linux readiness (节选)

```

// TcpEndpointFacts::Connection(connection) 分支 (续)
let mut events = PollEvent::empty();
let pending_error = connection.has_pending_error();

if pending_error {
    events |= PollEvent::ERROR;
}
if connection.is_terminal() {
    events |= PollEvent::HANG_UP;
}
if interests.contains(PollEvent::READABLE)
    && (connection.received_bytes() != 0
        || connection.is_local_read_shutdown()
        || connection.is_peer_receive_closed()
        || pending_error
        || connection.is_terminal())
{
    events |= PollEvent::READABLE;
}
if interests.contains(PollEvent::WRITABLE)
    && (connection.is_local_write_shutdown()
        || pending_error
        || connection.is_terminal()
        || (connection.connect() == TcpConnectFact::Connected
            && connection.send_capacity() != 0))
{
    events |= PollEvent::WRITABLE;
}
if interests.contains(PollEvent::READ_HANG_UP)
    && (connection.is_local_read_shutdown()
        || connection.is_peer_receive_closed()
        || connection.is_terminal())
{
    events |= PollEvent::READ_HANG_UP;
}
events

```

代码 32 连接事实继续投影为 READABLE、WRITABLE 与 READ_HANG_UP (节选, 续)

这段投影也说明了为什么我们不让后端直接返回 `POLLIN` 或 `POLLOUT`。同一份 TCP 事实需要服务普通 `read/write`、连接等待、接收连接等待、`poll/select` 和 `epoll`；每种操作关心的就绪条件不同。内核保存 Linux 对外承诺，协议栈保存协议真相。等待路径先读取快照，再登记唤醒通知，并在真正休眠前和醒来后重新读取一次，从而避免事实恰好在“检查”和“登记”之间改变而丢失唤醒。

网络端点的生命周期同样遵循这条边界。新 `fd` 放入进程描述符表之前发生失败时，创建保护对象会回滚尚未发布的网络端点；发布之后，`dup / fork` 共享同一个打开文件对象，只有最后一个 `fd` 引用被移除时才触发一次最终释放。Socket 适配层先禁止新的操作并撤销等待登记，再把非阻塞的 `release` 命令交给 TCP 模块；`FIN`、`RST`、`TIME_WAIT` 和协议引擎中的最终回收可以继续推进，不会让 `close()` 等待网络握手。这里和 Linux 的设计一致，从而兼容了一些特殊的语义。

8.6 后端替换边界

四层架构让协议后端成为一个有明确范围的实现选择。未来采用另一套 TCP/IP 引擎时，我们仍需适配网络接口、网络端点、缓冲区、定时器和协议推进方式；不同协议引擎的能力差异也需要认真处理。稳定边界已经把影响范围收窄：Socket ABI、`fd` 生命周期、`poll/select/epoll`、逻辑网络接口、IPv4 路由与选源策略，以及 `device/net` 的帧交接都不依赖 `smoltcp` 对象。

这项收益对当前代码的价值是：我们可以独立审查协议栈是否泄漏后端对象句柄、Socket 是否复制协议状态、驱动标识是否越过帧接口；任何一类穿透都有清楚的搜索范围和状态属主。后续增加协议或诊断能力时，我们先决定事实属于哪一层，再选择需要暴露的最窄 `Command`、`Event` 或 `Snapshot`。这些设计也许略显繁琐，但是我们换取了清晰的软件工程边界和可控的测试范围。

8.7 深入真实协议路径的主机测试

`Object fence` 还提供了一条强大的测试接缝。`FrameProvider` 没有绑定 `VirtIO`、`QEMU` 或内核线程，因此主机测试可以实现一个资源数量受限且行为确定的 `BoundedProvider`。测试运行的仍然是正式 `anemone-smoltcp-stack` 和真实 `smoltcp` 协议引擎，只把设备侧资源、完成顺序与链路事件换成测试能够精确控制的模拟设备。

下面这个测试准备三个待发送数据包，但模拟设备只有两个发送槽。第一次 `pump` 必须准确停在资源耗尽处；没有完成回收时再次 `pump`，不能凭空制造进展，也不能进入忙轮询。测试代码回收一个发送槽并发布重新检查通知后，第三个数据包才能继续提交。

```

impl FrameProvider for BoundedProvider {
    type RxToken<'a> = DeterministicRxToken<'a>;
    type TxToken<'a> = BoundedTxToken<'a>;

    fn transmit(&mut self, now: Instant)
        → TransmitOutcome<Self::TxToken<'_>>
    {
        self.observed_at = Some(now);
        if self.facts.link_state ≠ LinkState::Up {
            return TransmitOutcome::LinkUnavailable;
        }
        let Some(index) = self.tx.iter()
            .position(|lane| lane.slot == TxSlot::Available)
        else {
            self.normal_exhaustions += 1;
            return TransmitOutcome::Exhausted;
        };

        self.tx[index].slot = TxSlot::Reserved;
        TransmitOutcome::Ready(BoundedTxToken {
            lane: &mut self.tx[index],
            submission_log: &mut self.submission_log,
            consumed: false,
        })
    }
}

```

代码 33 BoundedProvider 用有限 TX slot 建模资源取得与正常耗尽（节选）

```

#[test]
fn deterministic_tx_exhaustion_preserves_and_resumes_socket_work()
{
    /* 地址配置与邻居表预热略。 */
    let mut stack = Stack::new();
    let mut provider = BoundedProvider::with_mac(LOCAL_MAC, 2);
    let interface = stack.add_interface(
        &mut provider,
        EthernetAddress::new(LOCAL_MAC),
        Instant::ZERO,
    );

    let packets = [
        build_raw_ipv4_packet(LOCAL_IP, PEER_IP, 1),
        build_raw_ipv4_packet(LOCAL_IP, PEER_IP, 2),
        build_raw_ipv4_packet(LOCAL_IP, PEER_IP, 3),
    ];
    let packet_refs =
packets.iter().map(Vec::as_slice).collect::<Vec<_>>();
    stack.queue_ipv4_for_host_validation(interface,
&packet_refs).unwrap();

    let exhausted = stack.pump(
        interface,
        &mut provider,
        Instant::from_micros(1),
        PumpBudget::new(1, 3),
    ).unwrap();

    assert_eq!(provider.submissions(), 2);
    assert_eq!(provider.live_tx(), 2);
    assert_eq!(provider.normal_exhaustions(), 1);
    assert!(exhausted.work_remaining);
    assert_eq!(exhausted.recheck, Recheck::Idle);
}

```

代码 34 主机测试构造三个数据包与两个发送槽，并验证首次资源耗尽（节选）

```
// 同一测试在资源耗尽后的连续断言（续）
let still_blocked = stack.pump(
  interface,
  &mut provider,
  Instant::from_micros(2),
  PumpBudget::new(1, 3),
).unwrap();
assert_eq!(provider.submissions(), 2);
assert!(still_blocked.work_remaining);
assert_eq!(still_blocked.recheck, Recheck::Idle);

provider.complete(0);
provider.publish_recheck();
assert!(provider.take_recheck());

stack.pump(
  interface,
  &mut provider,
  Instant::from_micros(3),
  PumpBudget::new(1, 3),
).unwrap();
assert_eq!(provider.submissions(), 3);
assert_eq!(
  provider.submitted_frames().iter()
    .map(|frame| raw_ipv4_marker(frame))
    .collect::<Vec<_>>(),
  [1, 2, 3]
);
```

代码 35 主机测试验证未完成回收时保持阻塞，并在重新检查通知后恢复推进（节选，续）

同一套模拟设备还覆盖接收帧注入、链路启停、完成回收次序、重复通知合并、未消费 token 的自动归还、超长帧拒绝、回调异常退出和多接口隔离。很多故障在 QEMU 中只能依赖偶然时序触发，在主机测试中可以变成每次都执行相同状态转换的回归测试。我们认为这是四层架构的一个核心优势：我们的测试不仅覆盖局部的单点，还能测试整个状态机闭环。

8.8 只读网络诊断面

真实网络能力还需要可观察性。常用的 `ip` 工具并不通过 `ioctl` 读取一张简单配置表，而是使用 `AF_NETLINK` 和 `rtnetlink` 请求链路、地址与路由的完整列表。我们实现了 Netlink Socket 传输、多段回复、请求序号、`NLMSG_DONE` / `NLMSG_ERROR`、阻塞与非阻塞接收、`MSG_PEEK` 与 `MSG_TRUNC`，再把请求转发给维护相应事实的模块。

诊断适配层在每个请求到来时取得一份独立快照：逻辑接口模块提供 `ifindex`、名称与接口种类；`device/net` 提供 MAC、MTU 和当前链路状态；`Ipv4ControlPlane` 提供地址、直连路由与默认路由。适配层释放各模块的锁之后，再把快照编码为 Linux 用户接口格式。它不复制一份长期存在的全局网络状态，也不从 `smoltcp` 私有路由表或对象句柄猜测内核策略。

因此，`ip link show`、`ip address show` 和 `ip route show` 展示的是内核各状态属主在请求时刻提供的只读事实。诊断快照、编码后的 Netlink 数据报和分段读取位置都不能反向影响设备接入、路由选择或数据包处理。这条边界让工具获得真实信息，也保持了单一真相源。

8.9 真实工作负载闭环

表 4 真实网络程序覆盖的跨子系统路径

用户操作	贯通的主要系统能力
<code>apt install</code>	DNS、UDP/TCP Socket、阻塞与 readiness、时间、用户态 TLS、下载数据写入文件系统。
<code>git clone https:// ...</code>	DNS、TCP 长连接、非阻塞 I/O、用户态加密库、进程与持久化存储。
<code>ping</code>	ICMP raw socket、权限检查、IPv4 路由与源地址选择、外部帧路径与回复接收。
<code>ip link/ address/route show</code>	<code>AF_NETLINK</code> 、 <code>rtnetlink</code> 多段回复，以及逻辑接口、网络设备和 IPv4 控制面提供的只读快照。

从这些工作负载回看四层架构，“网络已经连通”只是成果的一部分。Socket、协议状态、协议引擎和设备资源各自拥有清晰的 object fence；同一套语义既能接入真实网卡设备，也能在开发主机上接受细粒度、确定性的压力验证。我们为此付出了比直接嵌入 `smoltcp` 更多的设计工作，最终——得到了一套更容易解释、验证和继续演进的内核网络栈。

9 时间

Anemone 的时间子系统同时服务调度器、超时等待和用户态时间接口。我们既要维护不会倒退的单调时间，也要给 shell、编译工具和标准库提供真实日历时间；在此之上，软定时器、POSIX timer、timerfd 和 itimer 又以不同方式把未来事件交付给内核对象或用户程序。

9.1 时间线与 tick

Anemone 把架构相关的计时能力抽象成 clock source 和 clock event。clock source 负责读取单调递增的硬件计数，clock event 负责把下一次 timer interrupt 编程到指定 deadline。

内核启动时会把各 CPU 的硬件计数投影到同一条单调时间线上。`MonotonicInstant` 表示自系统启动以来不会因校时而倒退的时间，适合调度、timeout 和性能计时；`RealtimeInstant` 则在单调时间上叠加日历偏移，供用户态观察现实时间。我们用不同类型区分它们，避免把可调整的 realtime deadline 误当作稳定的单调 deadline。

系统启动时，RTC 驱动读取一次硬件日历，为 realtime 建立初值。QEMU 平台可以使用 Goldfish RTC，Loongson 2K1000 则使用 LS7A RTC provider。运行过程中，`clock_settime` 与 `clock_adjtime` 调整的是 timekeeper 中的 realtime 映射，不会破坏 monotonic 的连续性。

周期 tick 由 timer interrupt 推进。每次中断到来时，timekeeper 更新全局 tick 计数，并重新编程下一次中断。调度器、软定时器和若干用户可见时间接口都建立在这条单调时间线上。

9.2 软定时器

软定时器负责在未来某个时刻执行一次回调。Anemone 的 timer core 使用按 deadline 排序的 per-CPU 队列保存请求；timer interrupt 到来后，内核取出已经到期的请求，再根据事件选择的路径执行或投递回调。请求可以被替换或取消，因此文件关闭、重新设定 timer 和进程退出不必等待一个已经失效的旧回调继续修改对象。


```

enum TimerLane {
    Irq(Box<dyn FnOnce() + Send + 'static>),
    Threaded(Box<dyn FnOnce() + Send + 'static>),
}

pub unsafe fn schedule_local_irq_timer_event(
    expire: Duration,
    callback: Box<dyn FnOnce() + Send + 'static>,
);

pub fn schedule_threaded_timer_event(
    expire: Duration,
    callback: Box<dyn FnOnce() + Send + 'static>,
);

```

代码 36 软定时器事件显式区分 IRQ 和 Threaded 两种回调路径

我们明确区分了 IRQ 上下文的 callback 和线程上下文的 callback，允许消费者更加灵活地选择。

9.2.1 IRQ

IRQ 路径用于真正适合在 timer interrupt 中完成的短回调。到期事件从本 CPU timer queue 中弹出后，回调直接在中断上下文运行。等待超时这类路径需要和调度器 wait identity 竞争同一轮唤醒，因此继续保留在 IRQ 路径中，由等待核心负责区分事件唤醒、信号、强制唤醒和 timeout。

这一路径的好处是延迟短、路径直接，但它要求调用者非常清楚中断上下文的约束。Anemone 因此没有把所有 timer callback 都统一塞进这条路径，而是让每个调用点按自己的锁、生命周期和外部可见行为选择合适的完成位置。同时，IRQ 路径极易死锁或者竞态，这也是我们将接口标记为 unsafe 的缘故。

9.2.2 Threaded

Threaded 路径把到期检测和回调执行分开。时钟中断判断 deadline 是否已经到期，到期后只把回调投递到本 CPU 的 timer worker 队列，并唤醒对应内核线程；真正的对象状态推进、等待者唤醒和信号通知在内核线程上下文中完成。

这个拆分主要服务于对象定时器。timerfd 到期时需要推进 expiration counter、唤醒阻塞读者和 poll/select 等待者；进程定时器到期时需要更新线程组内的定时状态，并通过信号机制通知用户态。把这些动作从 timer interrupt 中移出后，对象可以在自己的锁下维护状态，再在合适的位置触发文件对象或信号路径。

9.3 POSIX Clock

Anemone 为用户态暴露了 POSIX 风格的 clock 框架。每个 clock 对象提供时间分辨率和当前时间查询；系统调用层根据用户传入的 clock id 找到对应对象，再把纳秒时间转换成 Linux ABI 使用的 `timespec` 形式写回用户空间。

```
pub trait Clock: Sync {  
    fn resolution_ns(&self) → u64 {  
        1  
    }  
  
    fn now_ns(&self) → u64;  
}
```

代码 37 `Clock` trait 统一了用户可见 clock 的查询接口

这套框架覆盖常用的实时钟、单调钟、粗粒度 clock，以及进程和线程 CPU 时间等 clock id。`clock_gettime`、`clock_getres`、`clock_settime`、`clock_adjtime` 和 `clock_nanosleep` 复用统一的时间来源。相对休眠与绝对 deadline 都能按指定 clock 解释，因此 libc 和应用程序不需要依赖内核私有接口。

9.4 用户可见定时对象

在 clock 和软定时器之上，Anemone 实现了 POSIX timer、`timerfd` 和 `itimer`。POSIX timer 支持创建、查询、重新设定和删除，既可以向进程发送定时信号，也可以按 Linux ABI 选择目标线程；周期 timer 会统计错过的到期次数。`timerfd` 则以匿名文件描述符暴露定时事件，支持读取到期次数、非阻塞访问和 poll/epoll 可读事件，适合被事件循环统一管理。`itimer` 沿用传统进程定时器接口，继续通过信号通知线程组。

它们都没有把自己的语义下沉到时间系统的基础设施层。软定时器只负责在合适的时间投递回调；文件对象的可读状态、到期次数、周期重排，以及进程定时器的 signal 投递和重新计时，都由各自对象在自己的状态锁下维护。这样，我们的时间子系统既能支撑 Linux 兼容接口，又保持了和 VFS、信号、调度等待路径之间清晰的职责分工。

10 架构抽象层

Anemone 的硬件抽象层设计与往届内核很不同的是：我们遵循**依赖反转**的原则，让使用方子系统定义自己需要的架构能力，再由具体架构实现这些能力，而不是让 `arch` 模块主动向全内核暴露一组预设接口。这样，HAL 的所有权留在内存、异常、调度、信号等使用方模块中，架构层只负责实现和重导出。

在 `arch` 模块，目标架构内部分别实现 `PagingArch`、`IntrArch`、`SchedArch`、`TrapArch`、`SignalArch` 等类型，`arch_select!` 根据编译目标选择 RISC-V 或 LoongArch 的实现；上层模块再通过这些别名访问自己定义的 trait 能力。

```
#[cfg(target_arch = "riscv64")]
pub use riscv64::{
    IntrArch, PagingArch, SchedArch, SignalArch, TimeArch,
    TrapArch, machine_init,
};

#[cfg(target_arch = "loongarch64")]
pub use loongarch64::{
    IntrArch, PagingArch, SchedArch, SignalArch, TimeArch,
    TrapArch, machine_init,
};

pub type PgDir = <PagingArch as PagingArchTrait>::PgDir;
pub type Pte = <<PagingArch as PagingArchTrait>::PgDir as
PgDirArch>::Pte;
pub type TrapFrame = <TrapArch as TrapArchTrait>::TrapFrame;
pub type SyscallCtx = <TrapArch as TrapArchTrait>::SyscallCtx;
pub type TaskContext = <SchedArch as SchedArchTrait>::TaskContext;
```

代码 38 `arch` 模块按目标架构选择实现，并把关联类型收束成统一别名。通过依赖倒置的设计，Anemone 的边界非常直接。内存管理只知道页表能力，调度器只知道如何保存和恢复任务上下文，异常路径只知道 trap frame 和 syscall register context，信号子系统只知道如何编码用户态 `ucontext`。不同 ISA 的寄存器布局、页表项格式、中断控制寄存器和返回用户态指令都被限制在架构实现内。

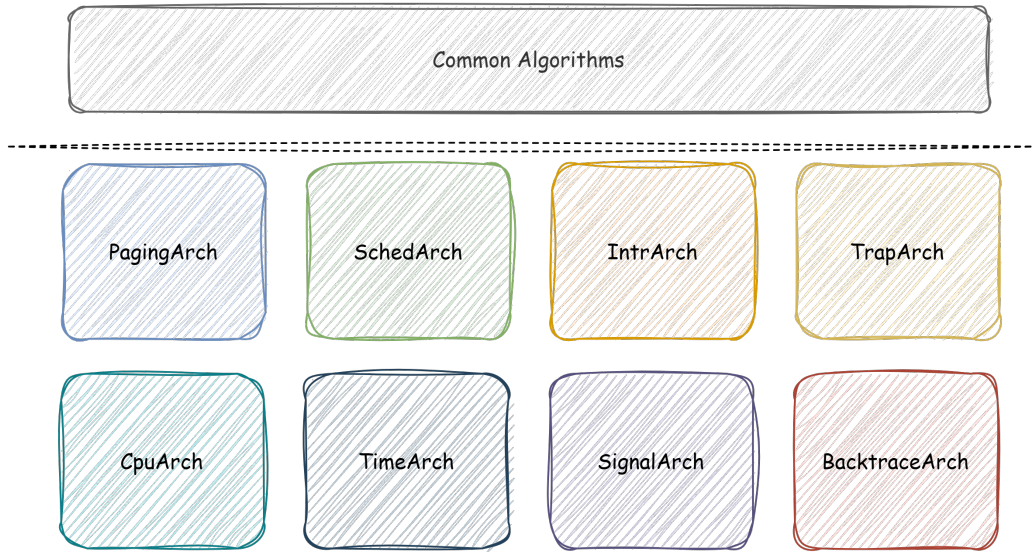


图 13 Anemone 定义了相当多的组件化的硬件抽象层

10.1 启动流程

Anemone 的启动流程分成两段。第一段是架构相关的早期入口，负责把 CPU 带到可以运行内核任务的状态；第二段是共享的 `bsp_kinit` / `ap_kinit`，负责进入普通内核初始化。这里我们只写到启动 `kinit` 为止，后续驱动发现、rootfs 挂载和第一个用户态进程属于其它章节的范围。

在 RISC-V 上，早期入口 `__nun` 清理临时寄存器，设置 bootstrap stack，写入 `satp` 启用临时页表，然后跳到 Rust 入口 `rusty_nun`。BSP 进入 `bsp_setup` 后清零 BSS、安装内核 trap 入口、注册 SBI early console 和基础电源处理器，从 FDT 早期扫描 CPU 数量、时钟频率和内存区域，初始化 per-CPU 区、物理页分配器、内核页表，并把 boot stack 重新映射到带 guard page 的区域。完成这些条件后，BSP 通过 SBI `hart_start` 唤醒 AP，创建 `bsp_kinit` 任务并切到调度器。

在 LoongArch 上，`__nun` 先配置 DMW、页表 walker 参数、TLB refill 入口和 bootstrap stack，再进入 `rusty_nun`。BSP 的 `bsp_setup` 同样完成 BSS、trap、早期控制台、FDT 早期扫描、per-CPU、物理内存和内核地址空间初始化，但 AP 唤醒通过 `csr_mail_send` 写入启动地址，再用 `IntrArch::send_ipi` 触发。LoongArch 还会在进入 `kinit` 前调用本架构的早期计时初始化，使本 CPU 的硬件计数源可用。

两个架构最终都创建内核任务来开始架构无关的第二阶段：BSP 创建 `bsp_kinit`，AP 创建 `ap_kinit`。从这一刻起，CPU 不再沿着裸启动栈继续推进初始化，而是作为普通内核任务进入共享路径。

进入决赛阶段后，这套启动边界同时覆盖了 QEMU virt、VisionFive 2 和 Loongson 2K1000 等平台。平台差异主要集中在早期页表、中断控制器、CPU 唤醒、计时源和设备

树交接；进入通用初始化后，调度、VFS、网络和用户程序启动继续走同一条路径。我们因此能够在扩展真实硬件时复用大部分内核，而不是维护两套彼此分叉的系统。

10.2 抽象层

下面几个 HAL 是 Anemone 当前跨 RISC-V 和 LoongArch 的关键接口。它们不是集中定义在 `arch` 目录，而是放在各自的使用方子系统中；本节介绍的每个接口都对应一组由该子系统真正使用的能力。

10.2.1 PagingArch

`PagingArch` 属于内存管理子系统。它定义架构必须提供的页表形状和硬件动作：页大小、页表层级、物理页号位宽，等等。页表遍历和映射算法可以共用一套通用实现（第四章提到的 `PageTable` 和 `Mapper`），但页表目录和页表项的二进制格式由 `PgDirArch` / `PteArch` 定义。

```

pub trait PagingArchTrait: Sized {
    type PgDir: PgDirArch;

    const MAX_HUGE_PAGE_LEVEL: usize;
    const PAGE_LEVELS: usize;
    const MAX_PPN_BITS: usize;
    const PAGE_SIZE_BYTES: usize;
    const PTE_PER_PGDIR: usize;

    fn setup_direct_mapping_region(pgtbl: &mut PageTable);
    unsafe fn activate_addr_space(root_ppn: PhysPageNum);
    fn tlb_shootdown(vpn: VirtPageNum);
    fn tlb_shootdown_all();
}

pub trait PteArch: Sized + From<u64> + Into<u64> + Copy {
    const ZEROED: Self;

    fn ppn(&self) → PhysPageNum;
    fn is_valid(&self) → bool;
    fn is_leaf(&self) → bool;
    fn is_branch(&self) → bool;
}

pub trait PgDirArch:
    Sized + Copy + Index<usize, Output = Self::Pte> +
    IndexMut<usize, Output = Self::Pte>
{
    type Pte: PteArch;

    const ZEROED: Self;
    fn is_empty(&self) → bool;
}

```

代码 39 `PagingArchTrait`、`PteArch` 和 `PgDirArch` 把通用页表算法与架构页表格式分开

通用页表代码可以根据 `PAGE_LEVELS`、`PTE_PER_PGDIR` 和 `PgDir` 索引规则完成查找、创建和释放；RISC-V 与 LoongArch 只需要在 `PteArch::new`、`flags`、`ppn` 等方法中解释自己的 PTE layout。

10.2.2 IntrArch

`IntrArch` 属于中断子系统。它的接口分成三类：读写本 CPU 的 IRQ flags，开关本地中断；发送和确认 IPI；初始化本 CPU 的本地 IRQ 入口。

```
pub trait IntrArchTrait: Sized {
    const ENABLED_IRQ_FLAGS: IrqFlags;
    const DISABLED_IRQ_FLAGS: IrqFlags;

    fn current_irq_flags() → IrqFlags;
    unsafe fn restore_local_intr(flags: IrqFlags);
    unsafe fn local_intr_enable();
    unsafe fn local_intr_disable();

    fn send_ipi(cpu_id: usize);
    unsafe fn claim_ipi();
    unsafe fn init_local_irq();

    fn local_intr_enabled() → bool;
    fn local_intr_disabled() → bool;
}

pub struct IrqFlags(u64);
```

代码 40 `IntrArchTrait` 用不透明的 `IrqFlags` 表示本地中断状态

我们把 `IrqFlags` 设计成不透明值，是为了让上层只能保存和恢复中断状态，而不能解释某个 ISA 的状态寄存器位。调度、锁和中断保护路径需要的是“保存当前状态、关中断、最后恢复”的能力，而不是 `sstatus` 或 LoongArch CSR 的具体布局。IPI 同样被抽象成 `send_ipi` / `claim_ipi`，使调度唤醒、TLB shutdown 或启动 AP 的路径不必知道底层是 SBI call 还是 LoongArch mail + IPI。

10.2.3 SchedArch

`SchedArch` 属于调度子系统，但它只负责最低层的寄存器上下文切换。调度队列、抢占策略、任务状态转换和地址空间切换都不在这个 trait 中。地址空间切换由内存管理通过 `PagingArch::activate_addr_space` 完成，`SchedArch::switch` 只保存当前上下文的 callee-saved registers，并加载下一个上下文。


```

pub trait SchedArchTrait {
    type TaskContext: TaskContextArch;

    unsafe fn switch(cur: *mut TaskContext, next: *const
TaskContext);
}

pub trait TaskContextArch {
    const ZEROED: Self;

    fn from_kernel_fn(entry: VirtAddr, stack_top: VirtAddr, args:
ParameterList) → Self;
    fn from_user_fn(entry: VirtAddr, ustack_top: VirtAddr,
kstack_top: VirtAddr) → Self;
    fn pc(&self) → u64;
    fn sp(&self) → u64;
}

```

代码 41 SchedArchTrait 只把任务上下文的保存与恢复交给架构层

10.2.4 TrapArch

TrapArch 属于异常和系统调用入口。它定义了两个对象：TrapFrame 是一次进入内核时保存的完整用户态寄存器现场，SyscallCtx 是系统调用分发需要的寄存器视图。这里，我们要求架构实现 syscall_ctx_snapshot 方法，它返回一个快照，可以用来需要重启系统调用或恢复信号前现场。


```

pub trait TrapArchTrait {
    type TrapFrame: TrapFrameArch;
    type SyscallCtx: SyscallCtxArch;

    unsafe fn load_utrapframe(trapframe: Self::TrapFrame) → !;
    fn syscall_ctx_snapshot(trapframe: &Self::TrapFrame) →
Self::SyscallCtx;
    fn restore_syscall_ctx(trapframe: &mut Self::TrapFrame,
syscall_ctx: &Self::SyscallCtx);
}

pub trait TrapFrameArch: SyscallCtxArch {
    const ZEROED: Self;

    fn sp(&self) → u64;
    fn set_sp(&mut self, sp: u64);
    fn set_tls(&mut self, tls: u64);
    fn set_scratch(&mut self, scratch: u64);
    fn set_arg<const IDX: usize>(&mut self, arg: u64);
    fn set_return_addr(&mut self, addr: u64);
}

pub trait SyscallCtxArch: Clone + Copy {
    fn syscall_arg<const IDX: usize>(&self) → u64;
    fn set_syscall_arg<const IDX: usize>(&mut self, arg: u64);
    fn syscall_no(&self) → usize;
    fn set_syscall_no(&mut self, sysno: usize);
    fn syscall_pc(&self) → u64;
    fn advance_syscall_pc(&mut self);
    fn syscall_retval(&self) → u64;
    fn set_syscall_retval(&mut self, retval: u64);
}

```

代码 42 `TrapArchTrait` 把 trap frame、syscall context 和返回用户态动作交给架构层

这组接口让 syscall 层只处理 Linux ABI 的分发语义，而不需要知道 RISC-V 用哪些寄存器传参、LoongArch 的 syscall number 放在哪里、异常返回地址应该如何推进。`load_utrapframe` 是最终返回用户态的架构动作，`advance_syscall_pc` 和

`set_syscall_retval` 则把 `syscall` 成功、失败和重启路径压缩成通用代码能调用的寄存器操作。

10.2.5 SignalArch

`SignalArch` 属于信号子系统。信号投递的策略、pending set、mask 和线程组选择由通用信号代码维护，但真正暴露给用户态的 `ucontext` 和 signal handler 入参依赖架构寄存器布局。因此我们把“如何把 trap frame 编码成 POSIX `ucontext`”“如何从 `ucontext` 恢复 trap frame”“如何让用户态下一步进入 handler”放进 `SignalArchTrait`。

```
pub trait SignalArchTrait {
    const MINSIGSTKSZ: usize;

    fn encode_ucontext(
        buf: &mut UContext,
        trapframe: &TrapFrame,
        mask: SigSet,
        altstack: linux_signal::SigStack,
        fpu: bool,
    );

    fn restore_ucontext(ucontext: &UContext, trapframe: &mut
TrapFrame, fpu: bool);

    fn prepare_trapframe_for_signal_handler(
        trapframe: &mut TrapFrame,
        signo: SigNo,
        handler: VirtAddr,
        sigframe_base: VirtAddr,
    );
}
```

代码 43 `SignalArchTrait` 负责用户态信号 ABI 中与寄存器现场相关的部分。这个拆分避免了两个常见问题。第一，信号核心不需要直接读写每个架构的通用寄存器数组；第二，架构层也不需要知道完整的信号投递策略。`RtSigFrame` 入栈、`rt_sigreturn` 恢复和 handler 入口布置都是用户可见 ABI 的一部分，但它们依赖的是 trap frame 形状，而不是调度器或 VFS 的内部状态。把它放在 `SignalArch` 中，正好把“信号语义”和“寄存器现场编码”分开。

10.3 用户访问与架构异常恢复

系统调用需要频繁访问用户指针。早期实现会先检查页表, 再通过内核指针读写用户内存; 但“检查时有效”并不代表真正访问时仍然不会缺页或被并发修改。决赛阶段, 我们改为由真实的 load/store 决定访问结果: RISC-V64 与 LoongArch64 分别提供短小的 copy 入口和异常恢复位置, 只有这些明确标记的访问发生 fault 时, 内核才把它转换成 EFAULT 或解析一次合法缺页。

通用的 `UserReadPtr`、`UserWritePtr` 和 `slice` 接口建立在这套架构能力之上。文件读写可以保留 fault 前已经完成的前缀, 要求事务完整性的系统调用也可以先 fault-in 输出区域。这种设计既更贴近硬件真实行为, 也让两个架构共享相同的 Linux 用户内存语义, 是 glibc、编译器和复杂系统调用能够稳定运行的重要基础。

10.4 架构专属能力

共享 HAL 并不意味着抹平 ISA 的特色。RISC-V64 实现了 `riscv_hwprobe` 与 `riscv_flush_icache`, 让用户态可以查询可用的基础指令扩展, 并在生成或修改代码后同步指令缓存; 全局 `membarrier` 则利用多核 IPI 为运行时提供内存顺序保证。

LoongArch64 侧, 我们补齐了 128-bit LSX 用户上下文。任务第一次使用 LSX 时, 内核为它启用完整向量寄存器保存, 此后的 syscall、中断、调度切换、fork/exec 和 signal frame 都能正确维持这份上下文。这项能力对现代编译器尤其重要: GCC 等程序会主动使用向量指令, 缺少完整上下文切换时往往不是立即报错, 而是以难以复现的数据损坏表现出来。

这些能力仍然遵循同一个原则: 通用子系统定义自己需要的语义, 架构层负责用本 ISA 的寄存器、异常和指令实现它。由此, Anemone 既能保持双架构主体共享, 也能让每个架构拥有真正可用而不是只有启动代码的专属支持。

11 构建系统

构建系统并不运行在内核中，却决定了一个内核项目能否被稳定地配置、构建、启动和验证。在不少教学或竞赛内核中，一份 Makefile 加上若干辅助脚本已经足以完成单一架构、单一平台的构建。但当项目扩展到双架构、虚拟机与真实开发板、多套用户环境和不同验证场景以后，平台常量、编译选项、磁盘路径和启动参数很容易散落在环境变量与脚本调用顺序中。此时，开发者虽然仍能“把内核编出来”，却很难准确回答这次构建最终描述的是哪一个完整系统。

我们把 Anemone 的构建系统设计成**从系统描述到可启动产物的类型化编排层**。内核功能、系统目标、硬件平台、用户程序、根文件系统和运行时输入分别建模，再由统一的解析器检查并组合这些事实。这样，Anemone 的可移植性从 HAL 接口延伸到了“如何稳定地产生并验证一个完整系统”这一工程交付边界。

11.1 从隐式脚本到显式系统模型

我们在设计构建模型时坚持一个基本原则：**每一类事实只由一个配置层拥有**。例如，物理内存起始地址属于平台，内核是否启用某项功能属于 KernelConfig，根设备和首个用户程序属于系统目标；它们不会因为某个脚本恰好需要这些值，就被复制到脚本参数或另一份配置中。图中给出了各层配置之间的主要关系。

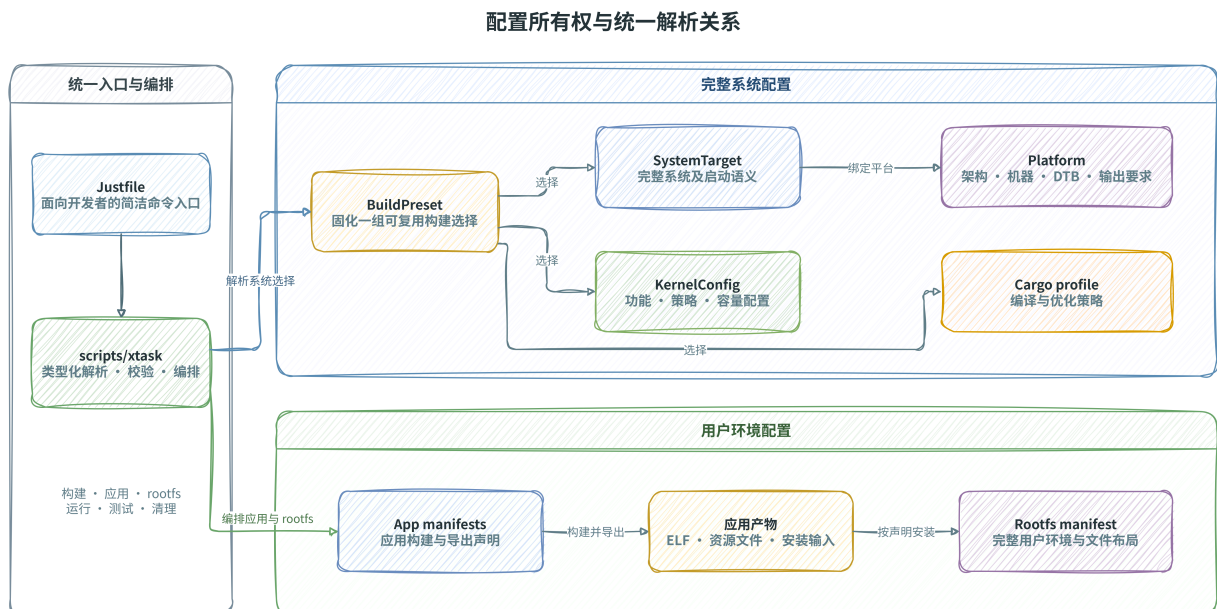


图 14 Anemone 构建配置的职责与组合关系。每种配置拥有不同事实，并由 xtask 统一解析和编排。

这套模型由以下几类对象组成。

- **KernelConfig** 描述内核功能、策略与容量参数，例如调试设施、文件系统支持、调度策略和各类资源上限。我们保留了内核开发者熟悉的 Kconfig 使用方式；目标平台、构建 profile 和宿主机路径由其它层负责。
- **Platform** 描述一个硬件或虚拟平台，包括目标架构、启动环境、物理地址常量、设备树交付方式、QEMU 机器描述，以及该平台要求发布的内核产物。RISC-V64、LoongArch64、QEMU virt、VisionFive 2 和 Loongson 2K1000 的差异都在这一层收束。
- **SystemTarget** 描述我们准备启动的系统：它选择 Platform，并确定根文件系统类型、根设备、静态网络部署和首个用户程序。相同平台因而可以承载普通 rootfs、性能环境或内嵌初始化程序等不同系统形态，而不需要复制平台配置。
- **BuildPreset** 把 SystemTarget、KernelConfig 和内核 Cargo profile 组成一个可复用的构建入口。preset 固化一套完整选择，任何字段都不会由其它隐藏状态补齐。
- **App manifest** 描述用户程序支持的目标架构、构建驱动和需要导出的产物；**Rootfs manifest** 则描述基础目录、需要安装的程序与文件，以及最终文件系统镜像。应用构建和 rootfs 组装由此也进入同一套可检查的交付链，省去了内核构建之外的手工准备过程。

这些配置用于解决规模扩大以后最容易出错的问题：哪些值应该共同变化，谁负责解释这些值，以及下游拿到的究竟是声明过的当前产物，还是某次旧构建偶然留下的文件。我们为关键配置定义了封闭的字段集合和解析类型，并在解析阶段检查跨层引用，从而尽可能在启动编译器和 QEMU 以前暴露错误。

11.2 单一入口与统一解析

Anemone 仓库使用 Justfile 提供简洁、可发现的统一入口，复杂逻辑则由 Rust 编写的 scripts/xtask 承担。我们可以通过 `just build`、`just app`、`just rootfs`、`just qemu` 和 `just test` 完成日常操作。这些入口共享同一套配置类型、路径解析与产物规则，无需分别维护 shell 逻辑。

一次普通构建可以写成：

```
just build --preset qemu-virt-rv64-release
```

代码 44 使用显式 preset 构建 RISC-V64 QEMU 系统

这条简短命令选择了一份完整的系统描述。解析器会得到确定的 SystemTarget、Platform、KernelConfig 与 Cargo profile，并报告本次选择的来源。对于不适合预先固定的场景，我们也允许调用者显式给出完整的 `target + kernel-config + profile`

元组。缺少其中一部分的选择会被拒绝，解析器也不会从“上一次构建结果”、开发者本地默认值或当前目录下恰好存在的文件中静默补齐。构建与 QEMU 使用同一个解析器，因此不会出现编译时选择了一个平台、运行脚本却又用另一套机器参数启动的平行配置。

我们还把宿主机相关的运行值与可复用配置分开。Platform 可以声明 QEMU 需要哪些绑定以及它们如何展开为参数；磁盘镜像、内核产物等具体路径由本次调用显式提供。这样，仓库可以记录“这个平台需要什么”，却不会把某位开发者机器上的绝对路径写进公共配置。缺失、重复或没有被当前动作消费的绑定都会在启动 QEMU 前被拒绝。

Rust 实现让 xtask 的构建配置与任务共享同一组枚举和结构体。架构、初始程序来源、应用构建驱动和设备树交付方式都必须经过类型化解析；跨字段检查可以给出带配置身份的诊断，单元测试也能覆盖选择、路径约束、参数展开和失败清理。Justfile 因此保持为面向人的薄入口，xtask 则成为配置解释和构建副作用的唯一编排者。

11.3 从配置到可启动系统

我们把构建成功定义为：所选系统需要的输入已经生成，内核已经编译，平台要求的公开产物也已完整发布。图中展示了一次构建与验证的主要阶段。

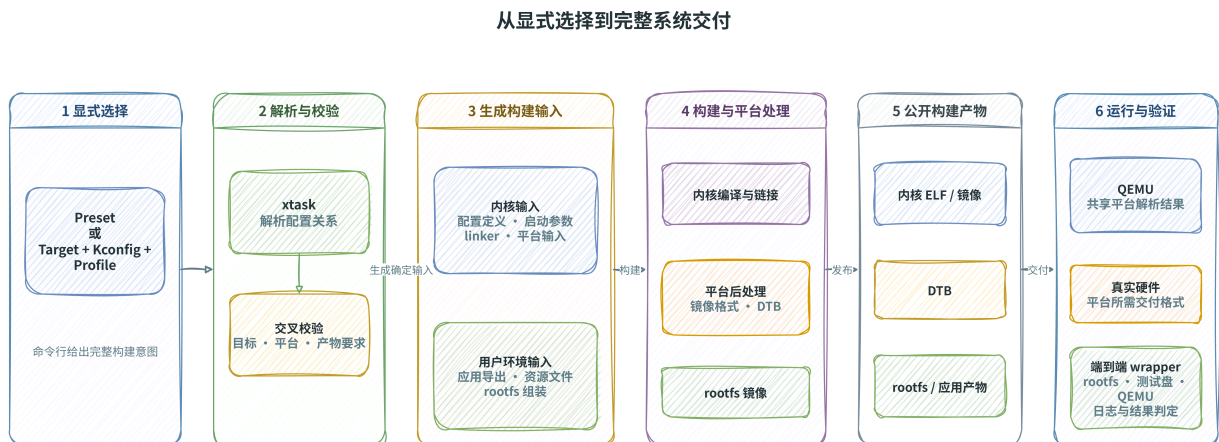


图 15 从显式系统选择到内核产物、运行平台和端到端验证的构建流水线。

xtask 首先解析完整选择并检查配置关系，随后为内核生成必要输入，包括 KernelConfig 对应的 Rust 定义、平台链接脚本、系统根挂载信息、静态网络配置和初始程序描述。内核源码不需要读取 TOML，也不需要知道 BuildPreset 或 SystemTarget 的存在，只消费构建系统生成的窄而明确的定义。这一边界让部署模型留在构建侧，同时避免在内核中建立第二套配置解析逻辑。

设备树同样属于平台交付的一部分。对于固件提供设备树的平台，我们不生成一份看似权威的副本；对于需要内嵌设备树的真实硬件平台，我们从受版本控制的规范 DTS 编译；对于由 QEMU 提供拓扑的场景，构建系统只使用机器类型、CPU、SMP 和内存等

平台拓扑生成本次构建的 DTB，而不会意外引入测试磁盘、网络 后端等运行参数。设备树由此和平台来源保持一致，各类脚本无需重复推断来源。

用户程序也走统一的产物导出路径。每个 app manifest 声明自身支持的目标、构建方式和产物；rootfs 构建按清单调用相同的应用构建器，将产物安装到 staging tree，再生成 Ext4 镜像。如果系统选择内嵌 初始程序，内核构建同样复用这条应用导出路径，并检查结果确实是唯一、可执行的普通文件。我们直接 使用构建系统导出的产物，不从 Cargo target/ 目录猜测路径。build/apps 与 build/rootfs 中的结果 是面向后续动作的公开产物。

内核编译完成后，xtask 统一发布 build/anemone.elf、link map 和平台要求的附加格式。启用内核符号表 时，构建系统会执行发现与最终链接两遍构建，使嵌入内核的符号表与最终 ELF 相互核验；面向 U-Boot 的平台则按 Platform 描述执行 objcopy 和 mkimage，只在完整成功后留下最终镜像。这样，我们可以继续使用 Cargo、链接器、dtc 和 QEMU 等成熟工具，同时由 Anemone 自己定义它们之间的输入、输出和失败边界。

11.4 双架构与端到端交付

这套构建模型从一开始就服务于多架构，RISC-V 与 LoongArch 共享同一套编排流程。应用清单明确声明支持的目标，Platform 选择对应的 target specification、工具链与启动产物，SystemTarget 再选择根设备和首个用户程序。我们用同一条内核构建路径覆盖 RISC-V64 与 LoongArch64，并将 QEMU virt、VisionFive 2 和 Loongson 2K1000 的平台差异限制在各自配置与必要的后处理步骤中。

对于比赛测例和完整回归，我们在通用构建能力之上提供了仓库级端到端 wrapper。它们会选择明确的 rootfs manifest，重新构建用户环境，把调用者指定的只读测试盘复制为本次运行的工作副本，选择对应的 系统与 QEMU 拓扑，构建内核并启动虚拟机，最后把串口输出保存为日志；需要自动判定的测试链还会检查 完成标记。初赛、决赛、RISC-V64 和 LoongArch64 的输入因此能够被明确区分；每次成功结果都对应一条 可以重复执行的完整路径。

与常见的脚本式组织相比，这套系统的优势来自显式关系。构建目标由 preset 或完整选择元组确定，平台 事实集中在 Platform，用户环境由 app 与 rootfs manifest 描述，编译和 QEMU 共享目标解析结果。接入新 架构时，我们复用编排流程，只增加相应的架构与平台事实；端到端 wrapper 固定输入、产物和日志边界。

构建系统把通常散落在 Makefile、环境变量和启动脚本中的隐式知识，转化为职责明确、可组合、可检查且 可复现的系统描述。从配置到验证的稳定闭环，使我们能够在快速演进内核功能的同时持续维护双架构、多平台和多种用户环境，避免工程复杂度沉积为一组只有原作者才能正确使用的脚本。

12 总结与展望

12.1 主要工作成果

从初赛到决赛阶段，Anemone 围绕 Linux ABI 兼容、核心 OS 能力和双架构适配持续推进。我们已经形成了一个以进程 / 线程模型、调度器、虚拟内存、VFS、IPC、设备驱动模型、网络栈、时间子系统和架构抽象层为主体的宏内核。它不仅能够运行测例，也能够承载交互式 shell、网络工具、glibc 用户环境、Rust 工具链和复杂软件构建。

在设计上，我们尽量让每个模块拥有清楚的职责。内存管理中的 VMO 与写时复制、文件系统上的 VFS 对象模型和页缓存、设备驱动模型中的 bus / device / driver 分层，以及网络栈中 Socket、协议状态、协议引擎和设备资源之间的隔离，都体现了这种思路。Linux 兼容接口由通用机制支撑，也就不必退化成一组针对测例的特判。

决赛阶段给我们带来的最大变化，是系统开始接受真实工作负载的检验。大型编译会同时放大进程创建、调度、缺页、TLB、路径查找、页缓存、Ext4 和块 I/O 的成本；交互式 shell 会同时使用 session、process group、signal、TTY 和 wait；网络程序则会贯穿设备、协议栈、Socket、epoll 和诊断接口。我们围绕这些完整路径继续完善 Anemone，使各个章节中的机制真正组合成了一套可用的操作系统。

跨架构方面，Anemone 已经同时面向 RISC-V 和 LoongArch 组织启动、页表、中断、trap、上下文切换和信号现场。HAL trait 由使用方子系统定义，再由具体架构实现，使大部分内核代码能保持共享。这个边界也让我们在迁移平台、维护双架构行为和解释问题时，有一个比较清楚的分层依据。

我们还为项目建立了一套完整的构建与交付系统。内核配置、系统目标、硬件平台、用户程序和 rootfs 各自拥有明确的配置边界，再由 Justfile 与 xtask 统一解析和编排；同一套流程能够产生 QEMU、VisionFive 2 和 Loongson 2K1000 所需的内核与启动产物，并串联双架构端到端验证。它让 Anemone 的模块化与可移植性从源码结构延伸到了可复现的系统交付过程。

12.2 开发经验总结

这次开发让我们最深的体会是，做 Linux 兼容内核不能只看 syscall 名字，也不能只看测试输出。很多问题只有把 Linux man page、libc 行为、LTP 源码和内核实现放在一起看，才能判断用户态真正依赖的语义是什么。例如错误码、阻塞唤醒、信号中断、文件描述符继承和路径查找这类细节，往往会跨过多个模块，单独修改某个 syscall wrapper 很容易制造新的不一致。

并发和生命周期问题也促使我们更重视模块职责。等待队列、文件对象、页表页、设备私有状态和信号现场都需要有明确的状态归属；否则 bug 出现时，很难判断哪个对象应该负责推进状态、释放资源或唤醒等待者。我们逐渐形成的做法是：先确认状态由谁拥有，再决定需要暴露什么接口；对关键不变量使用断言尽早暴露问题；能构造最小复现时，优先用最小路径验证判断，而不是在完整测试集合里反复猜测。

双架构开发让我们认识到，抽象层不能只是把所有架构函数集中到一个目录里。更好的方式是让上层子系统说清楚自己需要什么能力，再由架构实现这些能力。这样既能减少 RISC-V 和 LoongArch 之间的重复，也能避免架构细节反向污染调度、内存、信号和设备模型。

AI 工具在开发中主要帮助我们阅读材料、整理候选方案、做代码审查和改善文档表达，但所有结论都必须回到源码、测试和人工判断。我们把 AI 输出纳入工程流程，而不是让它替代设计决策；真正进入内核和报告的内容，需要经过队员理解、修改和验证。

12.3 未来计划

网络能力已经从初赛阶段的规划项发展为能够运行真实程序的完整路径。后续我们会沿着现有四层架构继续扩展动态控制面、诊断能力、协议覆盖和真实硬件网卡，同时保持 Socket、协议引擎与设备资源之间清楚的边界。

Linux ABI 的长尾语义仍然需要继续推进。未来我们会围绕文件系统一致性、ioctl、信号与等待交互、进程组 / session、资源限制和更多组合场景继续补齐行为，并用 LTP、真实用户程序和自构造压力测试共同验证。这里的目标不是简单增加 syscall 数量，而是让已有接口在跨模块交互时也保持稳定。

平台方面，我们会继续完善 RISC-V 和 LoongArch 的真实硬件适配，补充更多外设驱动和中断控制路径，并在多核场景下加强负载分配、锁粒度和长期运行稳定性。Anemone 当前已经具备清晰的模块边界和双架构基础，后续工作会在这个基础上继续扩展系统能力，而不是推倒已有结构重来。

A AI 使用情况

在 Anemone 的开发过程中，我们使用了 AI 辅助工具，但没有把核心判断交给 AI。我们的基本原则是：AI 可以帮助我们更快地阅读材料、整理候选方案、生成重复性代码和发现可能的问题，但内核设计、ABI 取舍、并发语义和最终合入必须由队员理解、审查和验证。

Anemone 的开发路线很有阶段性。开发前三个月，我们几乎完全依靠手写代码、阅读 Linux / LTP 行为和自己调试来推进内核。到了最后一个月，面对大量彼此关联的功能与性能工作，我们开始系统学习和使用 AI 辅助开发。这个过程让我们逐渐意识到，AI 生成内容本身不等于低质量；真正决定质量的是我们是否能给出清楚的上下文、是否能识别错误方向、是否能把输出放回源码和测试中验证。

我们主要使用的工具包括 ChatGPT 5.4 / 5.5、DeepSeek V4，以及 Codex CLI、Claude Code。不同工具的使用方式略有差异，但在我们的流程里，它们都只是辅助工具：提出候选解释、生成初稿、做局部审查或整理材料，没有替代队员对 Anemone 机制本身的判断。

A.1 使用范围

我们使用 AI 的场景主要有四类。第一类场景是阅读规范、代码和测试。实现 Linux 兼容接口时，仅凭 syscall 名字很容易误判语义；我们会让 AI 帮助整理 man page、Linux 行为、LTP 测例和 libc 调用习惯之间的关系，再由队员回到源码或测例中核对。这类工作需要处理大量材料，利用 AI 可以显著缩短搜索和整理时间，但我们不会直接把 AI 的解释当作结论。

第二类场景是生成候选实现和重复性代码。比如一些简单的文件对象、测试程序骨架、错误码分支或报告初稿，AI 可以根据我们给出的示例快速生成相似结构。我们通常会先手写一版能表达真实语义的代码或文字，再让 AI 补齐相似部分。

第三类场景是调试分析和代码审查。我们会把日志、失败现象或 diff 摘要交给 AI，让它列出可能的风险点，再由人工判断哪些问题成立。这个过程比较适合发现遗漏的路径、边界条件和不一致的接口，但 AI 也经常给出听起来合理、实际不符合当前代码结构的建议，所以我们不会让它直接决定修复方案。

决赛阶段的性能工作也使用了 AI 辅助。AI 可以帮助我们从观测结果中提出候选瓶颈、整理对照实验和检查优化是否可能改变正确性；但一个候选只有在真实工作负载中表现出稳定收益，并通过代码审查和功能验证后，才会进入正式内核。这样，AI 帮助我们扩大了搜索范围，却不会替代最后的工程判断。

第四类场景是文档表达。报告、RFC、transaction devlog 和小迭代记录中，AI 可以帮助我们吧口语化草稿整理成更清楚的工程说明。但每一段最终进入公开文档前，都要由队员检查是否符合实际实现，是否夸大完成情况，是否把私人调试材料写成公开事实，以确保团队合作的边界。

A.2 生成内容范围

我们把 AI 适合参与的内容限制在几个范围内：重复性样板代码、局部候选实现、测试程序思路、日志分析摘要、代码审查清单和文档润色初稿。这些内容有一个共同点：它们可以被人工快速核对，也能通过编译、测试、源码审查或文档事实检查来验证。

我们不会直接接受 AI 对核心架构的决定。进程拓扑、调度等待协议、虚拟内存对象、VFS 与设备模型边界、信号现场恢复、跨架构 HAL 这类设计，都需要队员先明确 owner、状态来源和外部可见语义。AI 可以参与讨论，但不能替我们决定模块边界。

我们也不会直接合入未经验证的 AI 代码。只要涉及锁、生命周期、地址空间、用户指针、信号返回、页表维护或跨模块回调，我们都会把 AI 输出当作候选 patch，而不是最终实现。能否进入代码库，取决于人工 review、构建检查和对应测试，而不是取决于回答看起来是否完整。

A.3 人工审查与验证

我们使用 AI 时最重要的约束是人工审查。一个建议是否成立，首先要回到 Anemone 当前代码，而不是回到 AI 记忆中的某种通用内核结构。我们会检查它是否落在正确的 owner 里，是否扩大了不该扩大的接口，是否引入了新的状态副本，是否改变了 Linux ABI 可观察行为。

```
带 3 个 subagent 做完只读审查了。没有发现 Apollyon；草案方向基本成立，但还不能直接进入实现，至少有几个 Keter 需要先写回草案。

Keter

1. ready queue / no-allocation 协议没有闭合。草案要求 IRQ 只移动 event 到 threaded-ready queue 并唤醒 worker，但 live code 现在只有值型 TimerEvent + BinaryHeap，没有 ready node、ready credit、reserve/commit/rollback 协议：anemone-kernel/src/time/timer.rs:11。需要在草案里明确 lane enum、threaded event 节点归属、heap/ready queue 预留顺序和失败线性化点。

2. worker wake 也必须纳入 IRQ no-allocation 证明。KThreadHandle::wake() 走 Event::publish() / wake_enqueue(); 如果误走 remote wake，会进入 IPI allocation/wait path：anemone-kernel/src/exception/ipi.rs:101。阶段 3 应明确 timer IRQ 只能唤醒 cur_cpu_id() 对应 worker，并审计本地 wake 路径是否满足 IRQ 约束。

3. fallible schedule 与对象 armed publish 的线性化还不够可实现。当前 timerfd_settime() 先改 state、后 schedule：anemone-kernel/src/fs/timerfd.rs:668；ITIMER_REAL 也依赖不可失败 IRQ schedule：anemone-kernel/src/task/itimer.rs:45。草案需要固定一种 reserve/commit 模型：可失败准备不发布，最终 commit 不失败并与对象 state publish 在同一锁边界下完成。

4. ITIMER_REAL 周期 rearm 不能沿用当前 drop-and-recurse。现在 callback 校验 validness 后 drop(real)，再递归 set_real_itimer()：anemone-kernel/src/task/itimer.rs:65。threaded 化后必须写明：rearm 要在 itimer state/token 下重新校验和提交，不能让旧 callback 覆盖并发 set/cancel 的新 timer。

Euclid

- setitimer() old-value copyout 顺序需要变成阶段 5 的明确检查项。fallible schedule 后，ENOMEM 失败时是否已写了 old_value 会变成可见 ABI 差异：anemone-kernel/src/time/itimer/api/setitimer.rs:23。

- threaded API 的 local CPU 语义建议更硬：schedule 绑定调用 CPU，worker 绑定到期 CPU，跨 CPU 替换只靠 generation/validness 过滤晚到 event。

- backlog 诊断方向正确，但最好写明 high-water/threshold counter 的状态归属、锁和“不参与 drop/merge/backpressure 分支”的审计点。

确认成立
Late initcall 窗口、kthreadd 前置、CPU online 前置、per-CPU pinned kthread worker 方向都和 live code 对得上：anemone-kernel/src/main.rs:197、anemone-kernel/src/main.rs:202。草案也没有错误扩大 timerfd ABI 或把 timerfd04 拉进第一版。

验证：本次未改文件；确认 etc/plans/threaded-timer-event/* 被 /etc gitignore 覆盖；git diff --check 通过；三个新文件的 git diff --no-index --check -- /dev/null ... 无 whitespace 输出。未跑 build/LTP，符合 docs-only review 边界。
```

图 16 一次 AI 代码审查结果示例。我们把这类输出作为风险清单和审查入口，最终结论仍由队员结合源码、构建和测试确认。

图中展示的是一次 AI 辅助代码审查结果。它把问题分成较高风险和次级风险，并给出可能涉及的代码位置。我们不会直接把这类输出当作事实，而是把它当作审查清单：先判断问题是否真的存在，再决定是否需要修改草案、实现或验证计划。对于不成立的建议，我们会明确丢弃；对于成立的建议，也要经过人工修改和后续验证。

在较大的修改中，我们还会用文档和流程约束 AI 输出。`AGENTS.md` 记录了仓库级编码规则和边界要求；`skill` 文件把反复出现的工作方式沉淀为可复用说明；RFC、transaction devlog 和小迭代记录则用于约束实现范围、停止条件、验证方式和回写路径。这样，agent 不只是“会写代码的工具”，而是被纳入一个可审查的工程流程。

验证方式随任务不同而变化。代码修改至少需要构建、格式或空白检查；涉及 ABI 的修改需要和 Linux / LTP / libc 行为核对；涉及并发和生命周期的修改需要检查锁顺序、唤醒顺序、资源释放和失败路径。AI 可以帮我们列出需要检查的点，但检查是否通过必须来自实际命令、源码阅读和人工判断。

A.4 幻觉与纠正案例

AI 最大的风险是幻觉。第一类常见问题是误判 Linux 或 LTP 语义。它有时会根据 syscall 名称给出一个通用解释，却忽略 man page 的 NOTES、libc 包装层行为或 LTP 测例中的具体断言。遇到这种情况，我们会先停止按回答修改代码，转而阅读 Linux 行为、测例源码和当前内核路径，再决定问题到底在 syscall wrapper、对象语义还是测试环境适配上。

第二类问题是沿着过期代码或错误 owner 边界给出建议。Anemone 在开发过程中经历了多次模块调整，旧路径、旧抽象或半废弃代码有时仍然能被搜索到。AI 如果只看到片段，很容易建议在错误层次修补，比如把应该属于 VFS、调度或信号核心的状态塞进 syscall 层。我们的纠正方式是先确认当前调用路径和 owner，再决定 write set；如果建议要求越过模块边界，就必须重新审查设计，而不是为了让 patch 看起来小就强行兼容。

这些经历让我们形成了一个比较直接的判断：AI 能缩短定位和起草时间，但不能替代内核开发中最关键的因果判断。尤其是并发、生命周期、ABI 和跨架构适配问题，真正困难的部分不是写出几行代码，而是知道这些代码应该属于哪个对象、在什么不变量下运行、失败时如何恢复、用户态能观察到什么。

A.5 方法论反思

我们最开始使用 AI 时，方式比较简单：给一个 prompt，拿到一个回答，再人工修改。随着项目推进，这种方式很快暴露出问题。内核工程的上下文太长，单次对话很容易遗漏已经确定的约束；agent 如果不知道当前 write set、验证要求和模块 owner，也可能把一个局部问题改成跨模块扩散。

后来我们逐渐把经验沉淀到工程材料里：把仓库规则写进 `AGENTS.md`，把反复使用的审查和调试方法写成 `skill`，把较大的实现拆成 RFC、transaction devlog 或小迭代记录。这样做不是为了让流程显得复杂，而是为了让 AI 的参与可追踪、可审查、可回滚。我们希望 AI 成为开发中的“加速器”，而不是“驾驶员”；它可以帮助我们更快地搜索、总结、生成和检查，但 Anemone 的设计责任、验证责任和最终解释责任仍然在我们自己手里。

B 参考资料

B.1 外部参考

- Linux 6.6。Linux UAPI、进程与信号语义、VFS、设备驱动、网络、调度与各架构实现的主要参考。
- Linux man-pages。系统调用、POSIX 接口、errno 和用户可见行为说明。
- POSIX.1。进程、文件、信号、终端、IPC 与时间接口规范。
- Zircon。VMO、地址空间 backing object 与写时复制设计。
- smoltcp。TCP/IP 协议引擎及其 Socket 实现。
- lwext4。Ext4 磁盘格式与文件系统实现。
- RISC-V Privileged Architecture、SBI Specification。RISC-V 启动、异常、中断、页表和多核固件接口。
- LoongArch Reference Manual。LoongArch64 指令、CSR、异常、页表、FPU 与 LSX 机制。

B.2 往届内核

- rCore。中心化调度上下文与 Rust 内核实现经验。
- Chronix。BusyBox 用户环境与应用安装方式。